MASTER THESIS

SELF-ORGANIZING RECURRENT NEURAL NETWORKS

Prospects of Biologically Plausible Artificial Brain Circuits Solving General Intelligence Tasks at the Imminence of Chaos

Supervisor:
PROF. DR. GORDON PIPA

Author:
SARANRAJ NAMBUSUBRAMANIYAN

A thesis submitted in fulfillment of the requirements for the degree of Master of Cognitive Science in the

NEUROINFORMATICS & ROBOTICS RESEARCH GROUP
INSTITUTE OF COGNITIVE SCIENCE

Osnabrück, Germany

February 7, 2019

Harder the Luckier

Declaration of Authorship

I hereby certify that the work presented here is, to the best of my knowledge and belief, original and the result of my own investigations, except as acknowledged, and has not been submitted, either in part or whole, for a degree at this or any other university.

_____

signature

_____

city, date

I ACKNOWLEDGEMENTS

Dr. Prof. Gordon Pipa, Head of Neuroinformatics Lab, Institute of Cognitive Science, Germany

It started from a question I asked you in a final exam two years ago and you helped me solve it. I wanted to return that favor. Soon, I decided to build this network from scratch and developed it as thesis work. I hope, I presented something in this thesis that could be valuable to you.

Dr. Prof. Stefano Nolfi, Head of Laboratory of Autonomous Robots and Artificial Life, Italy

It was my privilege to work in your lab. Thanks for giving me the chance and being my inspiration. You made the platform for my research career.

Xuan Huang, University of Mannheim, Germany

My sincere thanks to you for helping me by sponsoring the hardware that support execution of code works developed for this thesis.

CONTENTS:

I Acknowledgements i

II Introduction

a) Motivation 1

b) Outline of Chapters 2

III Chapters

1. Fundamentals of Neurophysiology & Neuro-inspired computing

1.1 Biological Neuron 3
1.2 Spike generation 3
1.3 Synapses 4
1.4 Synaptic plasticity mechanisms 4
1.5 Hebbian learning 5
1.5.1 Hebbian and Anti-Hebbian synapses 5
1.5.2 Long Time Potentiation and Long Time Depression 5
1.6 McCulloch-Pitts formalizations of neurons 6
1.7 Reservoir Computing 8
1.7.1 Reservoir 8
1.7.2 Liquid state machines 8
1.8 Self-Organizing Recurrent Neural networks 9

2. Self-Organizing Recurrent Neural Networks: Properties and Biological Plausibility

2.1 Network parameter definitions 12
2.1.1 Excitatory and Inhibitory pool size 12
2.1.2 Connectivity 12
2.1.3 Unit threshold 12
2.2 Plasticity mechanisms 13
2.2.1 Spike- Timing Dependent Plasticity 13
2.2.2 Homeostatic plasticity 14
2.2.2.1 Intrinsic plasticity 14
2.2.2.2 Synaptic normalization 14
2.2.3 Intrinsic Spike- Timing Dependent Plasticity 14
2.2.4 Structural plasticity 15
2.3 Network state evolution 15
2.4 Characteristics of SORN models 16
2.4.1 Network morphogenesis 16
2.4.2 Synaptic weight distribution 17
2.4.3 Spike, spike count and spike rate and Inter-spike intervals 17
2.4.4 Correlation between Neurons 21
2.4.5 Poisson-like network activity 22
2.4.6 Spike Source entropy 24

2.5 Role of plasticity mechanisms in network dynamics and organization 24
2.5.1 Contribution of Synaptic scaling in network dynamics 24
2.5.2 Contribution of Intrinsic Plasticity 26
2.5.3 Contribution of iSTDP in network activity 27
2.5.4 Role of Structural plasticity in Network reorganization 28
2.6 Network Stability Analysis 28

3. SORN for Sequential Decision-Making Problems

3.1 Concepts of artificial neural networks 33
3.1.1 Perceptron 33
3.1.1.1 Activation functions 34
3.1.1.1.1 Sigmoid 34
3.1.1.1.2 SoftMax function 34
3.1.2 Network Topologies 35
3.1.2.1 Feed forward neural networks 35
3.1.2.1.1 Binary class model representation 35
3.1.2.1.2 Multi class model representation 35
3.1.3 Supervised learning algorithm 36
3.1.3.1 Cost function 36
3.1.3.2 Backpropagation 37
3.1.3.2.1 Gradient descent optimization 37
3.1.3.2.2 Learning rate 38
3.1.3.2.3 RMS Prop optimization 38
3.2 SORN for sequence prediction 39
3.2.1 Training data generation 39
3.2.2 Schematic representation of Network training 40
3.2.3 Results and analysis 40
3.3 Reinforcement learning
3.3.1 Markov Decision Process 42
3.3.2 Policy objective function and gradient 43
3.4 SORN for complex classic control tasks 45
3.4.1 Cartpole pole balancing: Problem statement and the environment 45
3.4.2 Training data and experiment pipeline 46
3.4.3 Results and analysis 47

IV Conclusion:

a) Research Questions Answered 52
b) Future Research 52

V Bibliography 53
VI Supporting Information 58

I INTRODUCTION

a) MOTIVATION

There is ongoing quest for understanding the ability of Self-Organizing Recurrent Neural networks in processing spatio-temporal patterns in external stimuli [23,32,34]. Till date, this class of networks are applied extensively to study the underlying principles that dominates self-organization in brain circuits. Experimental evidences also prove that these networks driven by synergistic combination of neuronal plasticity mechanisms under random inputs and noises can mimic the properties of biological brain [33]. Operating near the critical state that favors information processing, these networks also show self-adaptation to varieties of stimuli and maintain its dynamics under subcritical regime suitable for learning.

Studies [83,84] hypothesized that, brain-like recurrent networks exhibiting dynamics between order and chaos can support complex real-time computations by mapping input sequences into corresponding output sequences. Intriguingly, SORN which has outperformed static networks in sequence prediction, memory and counting tasks [23,32,34] with similar structural and dynamical behavior as networks employed in [83,84] has never been studied under complex time-series input. By assumptions, the potential reasons could be i) Initialization problem, ii) SORN's highly sensitive intrinsic dynamical behavior and iii) the collapse of the criticality signatures developed by plasticity mechanisms when triggered by spontaneous inputs [34]. In accord with above assumptions, the network might behave undesirably for unconditioned input stimuli. Hence, in prior deploying this network for complex tasks, it is necessary to investigate the impact of varieties of inputs and plasticity mechanisms on network stability. Therefore, SORN with three plasticity mechanisms [23] and SORN with five plasticity mechanisms [32] are build and simulated under input values sampled from varying distributions. Further, SORN proposed in this work has structural plasticity occurring at slower rate than the base model [32]. However, the simulated network produced similar statistical features as the base model except, convergence with small fluctuations in connections. In addition, the characteristics of these networks follows the neurophysiological properties found in neocortex.

To study the intricacies in extending these networks to solve complex tasks, two problems with varying complexities, a simple time-series prediction task and a complex sequential decision-making problem are used. The experiments consist of two phases; i) Simulation phase at which networks learn temporal pattern in the input under plasticity mechanisms and ii) Training phase, a linear output layer is trained to map the network states into target outputs using either gradient descent algorithm or policy gradient algorithms with respect to the task. During simulation phase, the stability of the networks is measured using Hamming distance while at training phase, it is measured as a function of their performance. Furthermore, network configuration settings and the training strategies are evaluated with respect to the problems considered.

Overall, this thesis discusses the principles of network organization and adaptation in SORN and present evidences for its consistency with biological findings. Further, it explores the scope of this network in solving benchmark problems of general-purpose intelligence. It also provides guidelines to find the optimal network and training parameter values to achieve desirable performance in such tasks.

b) OUTLINE OF CHAPTERS:

Chapter 1 provides a descriptive outline on the structure and basic functions of single biological neuron and investigate its significances as network of artificial neurons through McColloch-Pitts neuronal model. It also introduces the neuro inspired artificial networks that mimics the properties of biological neuronal circuits structurally and functionally. The structure of SORN will be studied from a theoretical perspective and the scope of such networks to understand the neuronal adaptation and reorganization mechanisms in brain. Overall, it presents the background knowledge required to formulate the concepts and methods that are used to build self-organizing recurrent neural network models.

As a follow up, Chapter 2 explains the mathematical and computational models of SORN in detail. This chapter begin with the definitions of hyper parameters, followed by mathematical equations that approximate combination of neural plasticity mechanisms responsible for self-organization and adaptation. Later, the role of each plasticity mechanisms in the network are studied with experimental results obtained during network simulation.

Chapter 3 begins with an introduction to the concepts of Artificial Neural Networks and Reinforcement Learning algorithms. It focuses the problem of integrating those concepts with SORN to solve simple sequence prediction to complex classical control problems. Meanwhile, the influence of plasticity mechanisms and the variety of inputs during simulation and training will be investigated against the network's overall performance. This chapter will also demonstrate how this network go wrong while performing such tasks under uncertain conditions.

Data Availability: The corresponding code is available at https://github.com/Saran-nns/PySORN_0.1

III CHAPTERS

CHAPTER 1

FUNDAMENTALS OF NEUROPHYSIOLOGY & NEURO-INSPIRED COMPUTING

1.1 Biological Neuron

Neurons are the fundamental building blocks of nervous system. They are the information processing units of brain responsible for receiving and transmitting information across various regions. Each neuron in the brain communicate with each other in the form of an electrical signal called action potential.

Morphologically, neuron has three major parts, i) Dendrites, which receive inputs from other neurons; ii) Soma or cell body, where action potential is generated when the inputs are large enough and iii) Axon, a projection from cell body that conducts the action potential of this neuron to other connected neurons through a small gap called Synapses. A model of biological neuron is shown in *Figure 1.1*

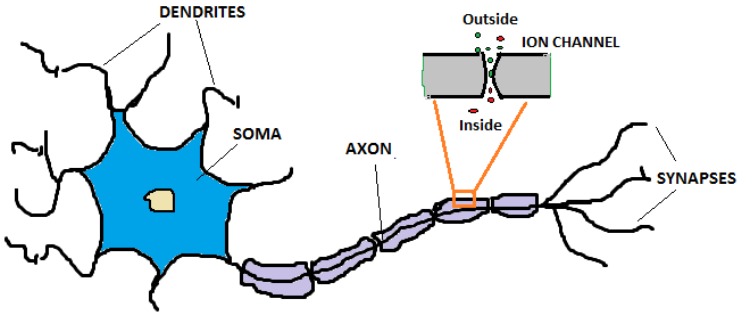


Figure 1.1: An overly simplified model of biological neuron with an example of Ionic channel

1.2 Spike Generation

Biochemically, neurons are filled with charged ions such as Na^+, Cl^- and K^+ which are enclosed within impermeable cell membrane. Ions are distributed across the cell membrane not in same quantity. Due to unbalanced ionic distribution, each neuron maintains a potential difference across cell membrane. The inner part of the membrane is negatively charged compared to the outer part of the membrane. This membrane potential at rest (in absence of external stimulus) is found to be -70mV and is called resting membrane potential or resting state. Ion channels otherwise gate keepers allow ions to flow in or out of neuron as shown in *Figure 1.1*.

When the neuron is induced by an external stimulus, ion channels open, which leads to movement of ions from higher concentration to lower concentration across membrane (ionic diffusion). Due to such ionic exchange between inner and outer surface of the membrane, potential difference across them gets reduced. This event is called depolarization of neuron. If the strength of the stimulus is increased, the membrane depolarizes further and at certain threshold, the outer side of

the membrane becomes negatively charged compared to the inner side. This change in membrane potential is called action potential or spike as shown in *Figure 1.2*.

1.3 Synapses:

Synapses are the connections between two neurons. When action potential of a neuron (presynaptic) reaches the synapses, chemical messengers called neuro transmitters are released through small vesicles from the neuron. This neuro transmitters will diffuse into the other neuron (postsynaptic) through receptors which will trigger changes in the membrane potential. Such presynaptic neuron induced membrane potential in post synaptic neuron is called Excitatory Post Synaptic Potential. Higher the frequency of incoming spikes, higher the number of neuro transmitters released and larger the depolarization in the post synaptic neuron [1].

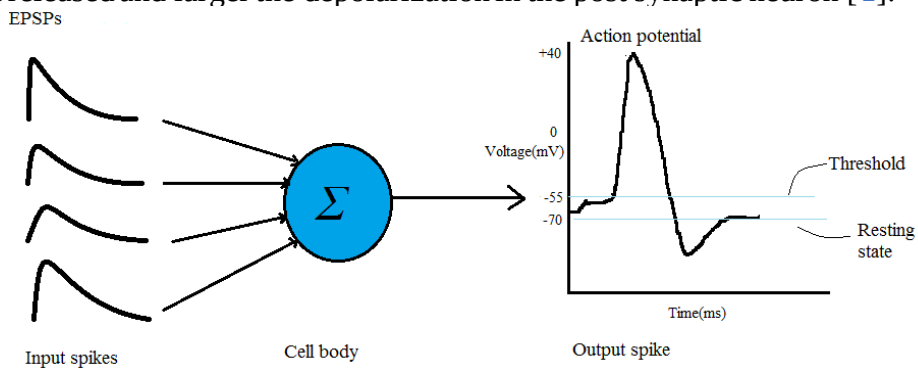


Figure 1.2 Action potential or spike generation mechanism

Synapses can be either electrical or chemical. In electrical synapses, pre and post-synaptic neurons are connected through a small gap junction, which allows action potentials to flow passively across membranes. Whereas in chemical synapses, there is no intercellular continuity. Therefore, no direct flow of action potentials between neurons. Neurons can exchange information through chemical messengers called neurotransmitters, which induces flow of ions through ionic channels as shown in *Figure 1.1*. However, the connections modality between neurons in SORN models are analogous to electrical synapses which we will discuss in the next sections of this chapter.

1.4 Synaptic Plasticity and Learning Principles

Brain is often referred as "plastic" because of neurons ability to reorganize both structurally and functionally in response to environmental stimulus. Neuronal reorganization is a direct implication of changes in synaptic connectivity strengths between neurons [2,3]. Such activity dependent variations in connection strengths is called synaptic plasticity. Several studies postulate that memory and learning are the outcomes of synaptic plasticity [3,4,5]. It is a widely accepted fact that synapses play the crucial role in formation and shaping of memory [6,7]. Hence, unravelling and understanding the functional mechanisms occurring in synapses could reveal the fundamental aspects of learning, memory storage and retrieval. One among the notable postulate on this discipline is Hebbian rule which explains adaptability of neurons during learning process [8].

1.4.1 Hebbian Learning

Hebbian learning is the process of activity dependent synaptic plasticity occurring between neurons. According to Hebb's postulate [8], two neurons are correlated when the pre-synaptic neuron is successful in activating the post synaptic neuron repeatedly. As a result, the process of increase in connection strength between these neurons occurs which further increases the efficiency of pre-synaptic neurons to influence the activity of postsynaptic neuron.

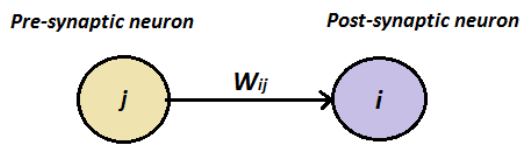


Figure 1.3:Uni-directed connectivity between two neurons

Consider, Y_i as activity of post-synaptic neuron i connected to pre-synaptic neuron j whose activity is represented as Y_j. Both these neurons are connected by synaptic strength W_{ij} (read as 'connection strength from neuron j to i') as shown in Figure 1.3 above. Then, the activity of neuron i, Y_i is given by the dot product of W and state of Y_j.

$$Y_i = W_{ij} Y_j \tag{1.1}$$

where $Y_j \in \{0,1\}$ and $W_{ij} \in \mathbb{R} \geq 0$

Then the change in weight W for a discrete process is given by Hebbian rule,

$$\Delta W_{ij} = W_{ij} + \eta\left[C\left(Y_i, Y_j\right)\right] \tag{1.2}$$

where, η is a learning rate constant and C is a correlation function between Y_i and Y_j.

Empirically, if correlation is negative, the weight W_{ij} will be decreased by small value η and if positive, the connection is strengthened by η. While in case of 0 correlation, there is no change in weight.

Therefore, the correlation function can be written as,

$$C\left(Y_i, Y_j, W_{ij}\right) = \left\{\begin{array}{l} 1 \ if \ \left(Y_i = 1 \ \cap \ Y_j = 1\right) \\ -1 \ if \ \left(Y_i = 0 \ \cap \ Y_j = 1\right) \ \cup \left(Y_i = 1 \ \cap \ Y_j = 0\right) \\ 0 \ if \ \left(W_{ij} = 0\right) \ \cup \left(Y_i = 0 \ \cap \ Y_j = 0\right) \end{array}\right\} \tag{1.3}$$

1.4.2 Long Time Potentiation and Long Time Depression

Correlated activity-based strengthening of connection weights between neurons using Hebbian rules leads to Long Time Potentiation (LTP) and Long Time Depression (LTD) [9]. LTP denotes increase in synaptic efficacy between highly correlated neurons above certain threshold during certain time interval. Whereas in LTD, synapses strength decreases below baseline value due to prolonged uncorrelated activity. Several studies also demonstrated clinical significance of LTP

and LTD for memory and learning [9-11]. To have physiological mechanism for memory, LTP should be persistent [12-15], input specific [15,16] and associative [15].

1.5 McCulloch-Pitt's Neuronal Model

The theory of artificial neural network is founded based on McCulloch and Pits model of biological neuron [17]. McCulloch Pitts neuron acts as a simple Threshold Logic Unit (TLU) as shown in *Figure 1.4a*. TLU respond to the input g based on a fixed threshold value or decision boundary θ set using the threshold function f that imitates 'all or none' activity [18]. Hence, based on strength of input(s), output from the neuron can be either 0 or 1 (*Figure 1.4b*). For the ease of understanding its characteristic properties, we will convert the biological neuron model (Figure 1.1) into a logical threshold gates proposed by McCulloch-Pitts.

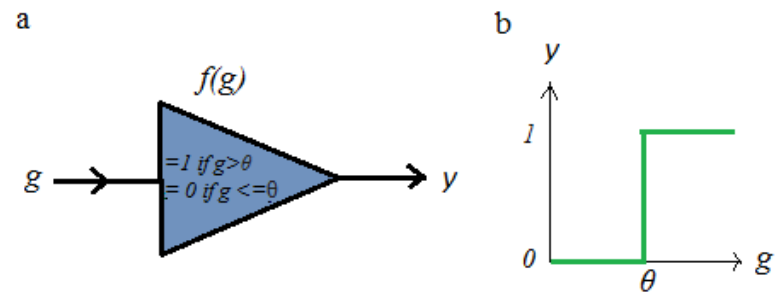


Figure 1.4: McCulloch-Pitts model of a neuron as a) Simple logic threshold gate and b) its response

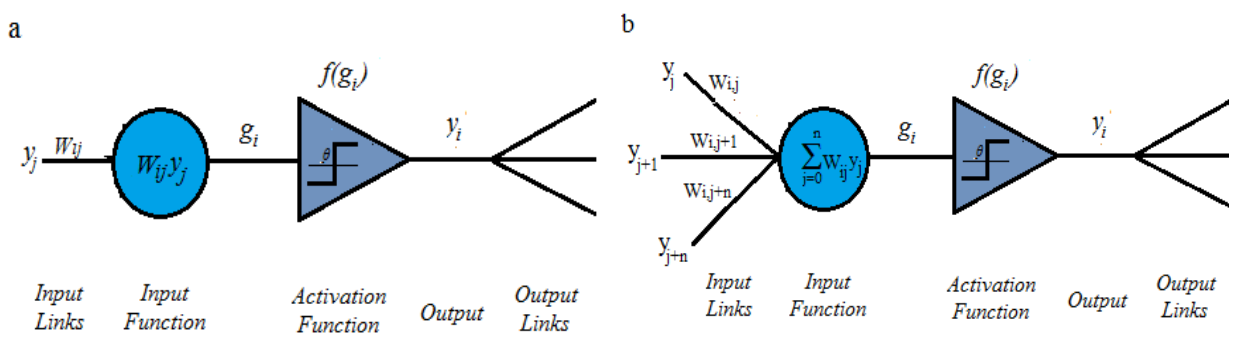


Figure 1.5: McCulloch-Pitts model of a neuron as a linear threshold gate. a) with single weighted input and b) with weighted sum of inputs

From the *Figure 1.5a*, the input to the McCulloch-Pitts unit i from neighboring neuron j, is given by,

$$g_i = W_{ij} y_j \qquad 1.4$$

where, $W_{ij} \epsilon (0,1)$ is the connection strength value from neuron j to i and y_j is the state (either 0 or 1) of neuron j.

However, as a unit in a network of neurons, it will receive inputs from multiple neighbors as shown in *Figure 1.5b*. Hence, the input to the neuron can be expressed through weighted sum as shown in Eq.1.5 below.

$$g_i = \sum_{j=0}^{n} W_{ij} y_j \qquad 1.5$$

According to McCulloch-Pitts formalization, a network of neurons is divided into two different groups, excitatory and inhibitory. Hence, each unit in a network receives inputs from two different pool of neurons as shown in *Figure 1.6*. Biologically, the inputs from excitatory pool of neurons will trigger positive changes in the membrane potential of a neuron which may generate spike. But, the inputs from inhibitory pool of neurons will counteract such positive impact from excitatory pool by inducing negative changes in membrane potential, thereby reduces the chances of spike generation in the neuron. Hence, we re-configure the neuronal model with multiple inputs (from *Figure 1.5b*) based on the above notion of inputs from two different group of neurons as in *Figure 1.6.*

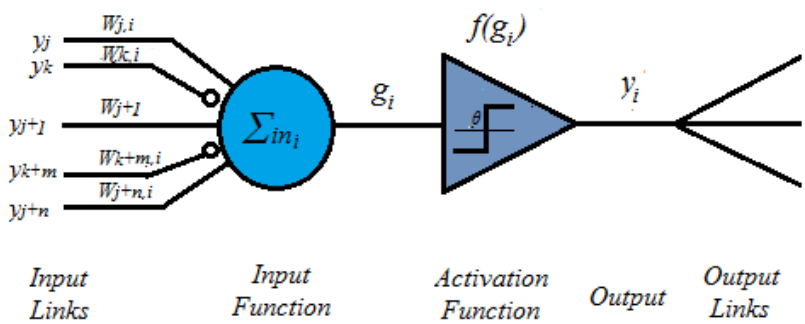


Figure 1.6: McCulloch-Pitts neuronal unit with inputs from excitatory and inhibitory pool of neurons.

With respect to the above model, input function defined using Eq 1.5 can be re-written as,

$$g_i := \sum in_i = \sum_{j=0}^{n} W_{ij} y_j - \sum_{k=0}^{m} W_{ik} y_k \qquad 1.6$$

In the equation above, the first part in the right-hand side represents the sum of inputs from n excitatory neurons and the second part denotes the sum of inputs from k inhibitory neurons. The negative sign between them indicates the negative influence in action potential of ith neuron by the inhibitory neurons.

Finally, the activity of the unit i is defined by a Heaviside step function, $f(g_i)$. Hence, in order to fire, the unit should receive total input value g_i greater than the internal threshold value θ, i.e.,

$$y_i := f(g_i) = \begin{Bmatrix} 1 \ if \ g_i > \theta \\ 0 \ if \ g_i \leq \theta \end{Bmatrix} \qquad 1.7$$

In this way, the McCulloch-Pitts neuron acts as linear threshold gate that maps the incoming signals into one of the two binary values. Many artificial neural networks we are using today are based on layers of such simplified model of neurons with randomly initialized weights between them and fixed threshold values based on the type of activation function applied on each neuron [19].

1.6 Reservoir Computing

Reservoir computing (RC) is a brain-inspired artificial neural net framework which consists of sparsely and recurrently connected McCulloch-Pitts neurons in a *liquid* called Reservoir. Based on the type of units in the reservoir, RC is classified as Liquid State Machines (LSM) [20] and Echo State Networks (ESN) [21]. In ESN, units are build using sigmoidal functions, whereas in LSM each neuron is constructed based on biologically inspired spiking neuronal model that resembles units in SORN.

1.6.1 Liquid State Machines

Structurally, LSM consists of a network of neurons sparsely and recurrently connected to each other in the reservoir. All units in the reservoir are then fully connected with a linear output layer as shown in *Figure1.7*.

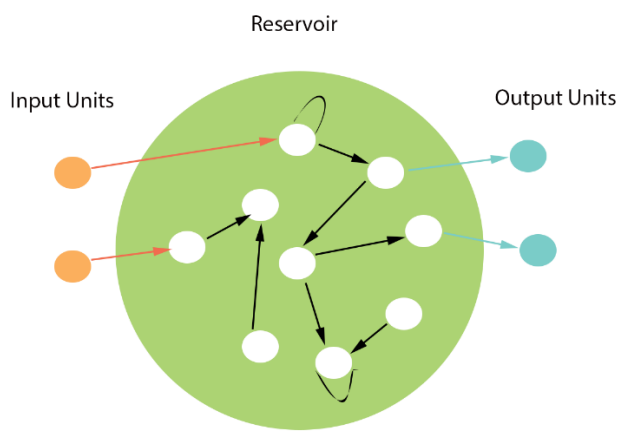


Figure1.7: Illustration of Liquid State Machine

An LSM consists of a reservoir with N neurons. Each neuron's activity or state can be denoted by a binary variable $x_i \in \{0,1\}$, where i stands for the ith neuron in the reservoir. Input to the ith neuron is a discrete-time variable u_i and its output is again a discrete-time variable y_i, $i \in \mathbb{N}$

For a network with k number of inputs for T time steps, the input matrix can be written as,

$$U = (u_1, \ldots . u_k) = \begin{bmatrix} u_{1,1} & \cdots & u_{k,1} \\ \vdots & \ddots & \vdots \\ u_{1,T} & \cdots & u_{k,T} \end{bmatrix} \; where\; k, T \in \mathbb{N} \qquad 1.8$$

The connection-strengths or weights between N neurons in the reservoir can be represented as:

$$W = \begin{pmatrix} w_{1,1} & \cdots & w_{1,N} \\ \vdots & \ddots & \vdots \\ w_{N,1} & \cdots & w_{N.N} \end{pmatrix} where \quad w \in \mathbb{R} := 0 \leq w \leq 1 \qquad 1.9$$

Hence, the state x of each neuron in the network at discrete timestep t is,

$$x_i(t) = f(g) \tag{1.10}$$

$$g = \sum_{j=1}^{N} w_{ij} x_j(t-1) + u_i(t) \tag{1.11}$$

where, $u_i(t)$ is the input received by the neuron i at time step t and f is any non-linear sigmoid function,

$$f = \left(\frac{1}{1+e^{-x}}\right) \tag{1.12}$$

or Heaviside step function,

$$f = \begin{pmatrix} 1 & if\ g > \theta \\ 0 & if\ g \leq \theta \end{pmatrix} where, \theta \in \mathbb{R} := 0 < \theta < 1 \tag{1.13}$$

Then, the output computed by the linear readout layer at time step t is,

$$v_i(t) = \sum_{j=1}^{N} w_{ij} x_i(t) \tag{1.14}$$

Schematically, SORN networks are built upon the similar architecture as LSMs. However, SORN differs from standard LSM models in the following aspects,

i) Reservoir is divided into two distinct pool: Excitatory and Inhibitory
ii) Self-connection or recurrent connections are not allowed.
iii) The neuronal connections in reservoir evolve over time and optimized by the neuroplasticity mechanisms whereas in LSMs the reservoir is fixed and only the connection weights between the reservoir and linear output layer is learned using optimization algorithms [20-22].

1.7 Self-Organizing Recurrent Neural networks

Self-Organizing Recurrent Neural (SORN) networks are a class of reservoir computing models build based on plasticity mechanisms in biological brain. Recent studies on SORN shows that such models can mimic neocortical circuit's ability of learning and adaptation through neuroplasticity mechanisms. Structurally, unlike other liquid state models, SORN consists of pool of excitatory neurons and small population of inhibitory neurons. First such network [23] was introduced with three fundamental plasticity mechanisms found in neocortex [24], namely Spike timing dependent plasticity (STDP), intrinsic plasticity (IP) and Synaptic scaling (SS). Spike Timing-Dependent Plasticity or Hebbian Learning with positive feedback (rapid cycle of synaptic potentials) selectively strengthens correlated synapses and weaken the uncorrelated. Such activity dependent rules lead to Long Time Potentiation (LTP) and Long Time Depression (LTD) [25].

Biologically, both LTP and LDP are assumed to possess substrates of learning and memory at the cellular level of neocortex [26]. However, in dynamical systems, such phenomena will drive the network either towards the state of bursting activity in case of LTP or towards state of attenuation due to LTD [27,29]. These destabilizing influences of STDP are counteracted by homeostatic plasticity mechanisms [28,29]. Homeostatic mechanisms are a set of negative feedback (action potential suppressing) regulatory mechanisms that scales incoming synaptic strengths and balances neuronal activity through synaptic normalization and intrinsic plasticity. Experimental evidences also prove that synaptic scaling found to balance the activity between excitatory and inhibitory neurons in-vivo [45].

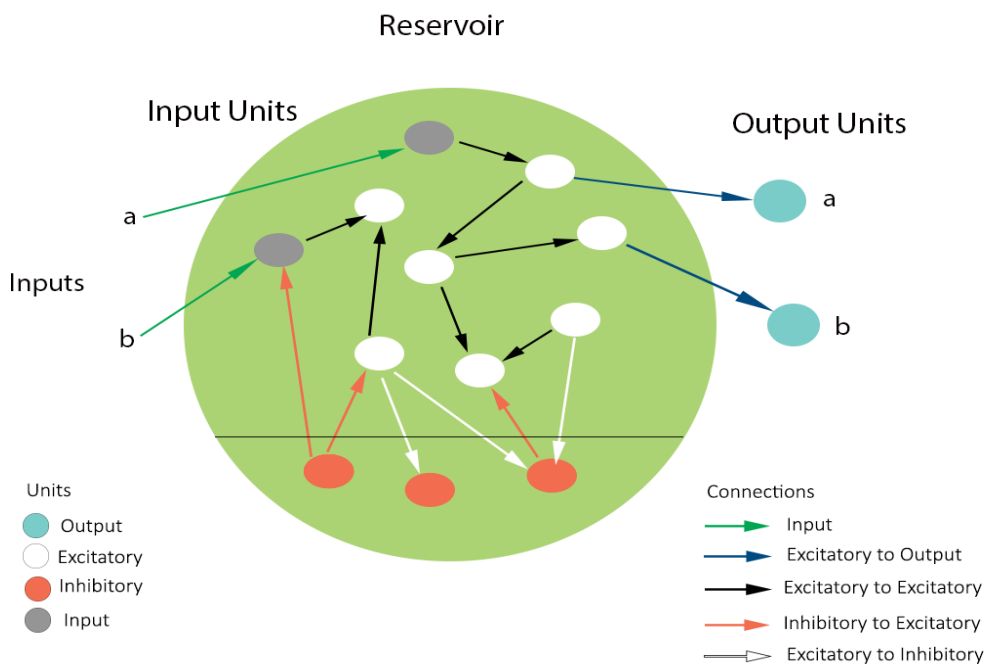


Figure 1.8: Illustration of Self-Organizing Recurrent Neural Networks

Together, they maintain the overall activity of network within subcritical range, despite the network being driven by positive feedback from fast Hebbian plasticity [28-31].

In recent proposed models [32-34], SORN is extended with two more plasticity mechanisms, inhibitory spike timing dependent plasticity and structural plasticity [32]. While connections between excitatory neurons (E-E) subjected to STDP rules [41-44], connections from inhibitory population to excitatory populations(E-I) are regulated by iSTDP [35,43]. Structural plasticity, generates new connections constantly at a smaller rate between unconnected synapses. Many studies argued that, such structural changes induce neuronal morphogenesis which leads to network re-organization with functional consequences over learning and memory [36-39].

The mathematical descriptions of plasticity mechanisms proposed in SORN [23,32] simplifies the structural and functional connectivity mechanisms that resembles information processing, learning and memory phenomena that occur in neuro-synapses of neocortex region. Recent

experimental evidences confirm that SORN outperforms other static reservoir networks in spatio-temporal tasks and maintains the dynamics of the network in subcritical state suitable for learning [32-34]. Further research on such network mechanisms unravels the underlying features of synaptic connections and network activity in real cortical circuits. Hence investigating the characteristics of SORN and extending its structural and functional attributes by replicating the recent findings in neural connectomics may reveal the dominating principles of self-organization and self-adaptation in neocortical circuits at microscopic level. Moreover, characterizing these mechanisms individually at that level may also help us to understand some fundamental aspects of brain networks at mesoscopic and macroscopic scales. Hence, this work will explore the plasticity mechanisms in neocortical circuits and briefly summarize and analyze the characteristic properties and network dynamics of SORN by reproducing the network models proposed in [23,32-34].

To summarize, this chapter introduced the physiology of individual and network of biological neurons with their corresponding mathematical models build based on spike generation mechanisms. Also, it presented artificial neural network models like LSM along with brief theoretical introduction into neuroplasticity inspired SORN concepts and its structure. The next chapter will investigate and construct the SORN network models based on neuronal plasticity mechanisms and analyze their characteristic properties along with their plausibility to biological findings up to date.

CHAPTER 2

SELF-ORGANISING RECURRENT NEURAL NETWORKS: PROPERTIES AND BIOLOGICAL PLAUSIBILITY

2.1 Network Parameter Definitions

2.1.1 Excitatory and Inhibitory Pool Size

The excitatory pool consists of N^E number of neurons, among which N^U are input neurons. Then the number of neurons in inhibitory pool is given by,

$$N^I = 0.2 * N^E \tag{2.1}$$

Each neuron in their respective pool are developed based on McCulloch Pitts neuronal model studied in Chapter1.

2.1.2 Connectivity

As shown in Figure 1.*8*, there are 4 types of connections in SORN,

i) From Excitatory to Excitatory(E-E) units and their corresponding strengths are represented as matrix, W_{ij}^{EE}
ii) From Excitatory to Inhibitory (I-E) units and their weights are denoted as W_{ij}^{IE}
iii) From Inhibitory to Excitatory (E-I) units and their weights as W_{ij}^{EI}
iv) From Excitatory pool to linear output layer (O-E) and their weights as W_{ij}^{OE}

Unlike LSMs, the self-connections or direct connections W_{jj}^{EE} or W_{jj}^{II} are not allowed in SORN.

In earlier model [23], the E-E connections are initialized as sparse and random, while the other connections E-I, I-E and O-E are dense. Whereas in recent models of SORN [32], the E-E and I-E connections are initialized as sparse and random and the E-I connections as dense. Further, all connection values are drawn randomly from either gaussian or uniform distribution.

The connection weights W_{ij}^{OE} is not an integral part of the reservoir and has no influence over the activity of neurons in the pool. This weight matrix is used while training the linear output layer to perform tasks. Therefore, the significance of this weight matrix along with optimization algorithms will be studied in detail from later chapters.

2.1.3 Unit Threshold

In conventional neural networks, the threshold values are implicitly fixed by the activation functions for each unit. In SORN, the initial threshold values for excitatory T^E and inhibitory units T^I are drawn uniformly from the interval [0,1] and are actively updated by plasticity mechanisms during network simulation.

2.2 Plasticity Mechanisms

2.2.1 Spike- Timing Dependent Plasticity

Though Hebbian theory characterizes neuronal mechanism for change in synaptic strengths, it failed to express the importance of temporal order in activity between neurons. Later, several studies investigated and found activity-driven modification of synapses based on precise timing of relative post-synaptic response to pre-synaptic activity [41-44]. Such activity driven and spike-timing based change in synaptic efficacy between excitatory neurons is called Spike-Timing Dependent Plasticity (STDP) [42-44].

The temporal evolution of synaptic efficacy between two neurons can be studied by unfolding their activity over certain time interval as shown in *Figure 2. 1*.

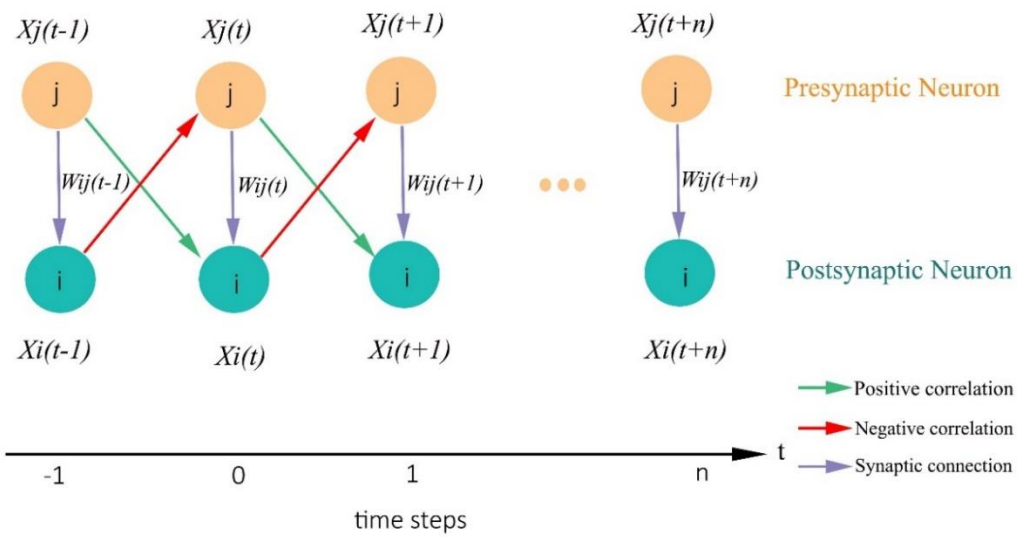


Figure 2. 1 Network of two connected excitatory neurons unfolded in time

Then the change in connection strength is given by STDP rule,

$$\Delta W_{ij}^{EE} = \eta_{STDP}(x_i(t)x_j(t-1) - x_i(t-1)x_j(t) \qquad 2.2$$

The connection weights between neurons *j* and *i* for the next time step can be updated as,

$$W_{ij}^{EE}(t+1) = \Delta W_{ij}^{EE} + W_{ij}^{EE}(t) \qquad 2.3$$

Through above equations, STDP regulates the synapses between excitatory units (W_{ij}^{EE}) by strengthening the synapse with a small fixed value η_{STDP} when unit *i* is active following the activation of unit *j* (positive correlation, green arrows in *Figure 2.1*). Similarly, it weakens the synapse by the same value when unit *i* is active before unit *j* (negative correlation, red arrows in *Figure 2.1*).

2.2.2 Homeostatic Plasticity

Homeostatic plasticity counterbalances the destabilizing influences of LTP and LTD induced by activity driven STDP. Thereby it helps to maintain the network in salient functional state suitable for learning [45]. There are two types of homeostatic plasticity mechanisms, Intrinsic Plasticity (IP) and Synaptic Scaling (SS) or Synaptic Normalization (SN).

2.2.1.1 Synaptic Scaling

Synaptic scaling normalizes the incoming connection strengths for each excitatory neuron at every timestep. It preserves the neuron from incoming elevated activity from its neighborhood.

$$W_{ij}^{EE}(t) \leftarrow W_{ij}^{EE}(t) / \sum_j W_{ij}^{EE}(t) \qquad 2.4$$

Also note that, this rule doesn't change the relative strengths of connections fixed by STDP but introduce competition between presynaptic neurons to contribute to the activity of postsynaptic neuron.

2.2.1.2 Intrinsic Plasticity

IP updates thresholds of excitatory units while the inhibitory thresholds are fixed at random initial conditions.

$$T_i(t+1) = T_i(t) + \eta_{IP}(x_i(t) - H_{IP}) \qquad 2.5$$

Where, η_{IP} is a small learning rate constant and H_{IP}=2 $\times N^U/N^E$ is the average firing rate for each neuron, in which number of input spikes (N^U) should be approximately half of total number of excitatory spikes (in N^E units) in the reservoir.

2.2.3 Intrinsic Spike-Timing Dependent Plasticity

The synaptic efficacies from inhibitory population to excitatory population of neurons are maintained by STDP-like plasticity mechanism called iSTDP [44]. In the earlier models of SORN, these connections are fixed with their random initial values while in recent models, they are initialized randomly with connection density 20% of the total number of possible connections between E-I and optimized under iSTDP rule.

$$\Delta W_{ij}^{EI} = -\eta_{inhib}\, y_j(t-1)\left(1 - x_i(t)\left(1 + \frac{1}{\mu_{IP}}\right)\right) \qquad 2.6$$

Therefore, the connection weights between inhibitory neuron j and an excitatory neuron i for the next time step can be updated as,

$$W_{ij}^{EI}(t+1) = \Delta W_{ij}^{EI} + W_{ij}^{EI}(t) \qquad 2.7$$

Through above equations, iSTDP regulates the synapses from inhibitory units to excitatory units W_{ij}^{EI} by weakening the synapse with a small fixed value η_{inhib} when unit *i* is inactive following the activation of unit *j* (positive correlation) and increases the synapse strength by the same value when unit *i* is active following the activation of unit *j* (negative correlation).

2.2.4 Structural Plasticity

Biologically, structural plasticity(SP) plays a vital role in learning and memory [46,47]. However, how it contributes and functionally influence the overall plasticity is not completely understood due to the complexity in the dynamics of interactions between excitatory and inhibitory neurons. Few studies found that synaptogenesis occurs in neural circuits at constant rate [48,49]. Recent study also proves that randomly added synapses can encode context in the input stimulus [50]. Similarly, structural plasticity in SORN generates new connections randomly between unconnected pair(s) of excitatory neurons at probability of 0.1.

2.3 Network State Evolution

Using plasticity mechanisms, the evolution of state of each individual neuron in the excitatory pool in the recent SORN model is given by,

$$x_i(t+1) = \Theta\left(\sum_{j=1}^{N^E} W_{ij}^{EE}(t)\ x_j(t) - \sum_{k=1}^{N^I} W_{ik}^{EI}(t)\ y_k(t) + u_i(t) - T_i^E(t) + \xi_E(\mathrm{t})\right) \quad 2.8$$

where,

Θ - Heaviside step function

$\sum_{j=1}^{N^E} W_{ij}^{EE}(t)\ x_j(t)$ - Sum of incoming drive (action potentials) from neurons in excitatory pool to neuron *i*

$\sum_{k=1}^{N^I} W_{ik}^{EI}(t)\ y_k(t)$ - Sum of incoming drive from neurons in inhibitory pool to excitatory neuron *i*

$u_i(t)$ - Input

$T_i^E(t)$ - Threshold of the unit *i* in excitatory pool

$\xi_E(\mathrm{t})$ - White gaussian noise applied to each neuron in the pool

The evolution of inhibitory neuronal states is decided by,

$$y_i(t+1) = \Theta\left(\sum_{j=1}^{N^E} W_{ij}^{IE}(t)\ x_j(t) - T_i^I + \xi_I(\mathrm{t})\right) \quad 2.9$$

where,

$\sum_{j=1}^{N^E} W_{ij}^{IE}(t)\ x_j(t)$ - Incoming action potentials from excitatory neurons

T_i^I - Threshold of unit i in inhibitory pool

$\xi_I(\mathrm{t})$ - White gaussian noise

Furthermore, the recurrent activity of the network without external stimulus can be derived from Equation 2.7 as

$$R_i(t+1) = \Theta\left(\sum_{j=1}^{N^E} W_{ij}^{EE}(t)\ x_j(t) - \sum_{k=1}^{N^I} W_{ik}^{EI}(t)\ y_k(t) - T_i^E(t) + \xi_E(\mathrm{t})\right) \qquad 2.10$$

Therefore, we can define the pseudo state of the network using the recurrent activity as,

$$x_i'(t) = \theta\big(R_i(t)\big) \qquad 2.11$$

The earlier SORN model differs from the recent ones in following factors, i) only three major plasticity mechanisms (STDP, IP and SN) ii) W^{IE} connections are dense and fixed at their initial values and iii) Neuronal state measure doesn't include noises.

2.4 Characteristics and Biological Plausibility of SORN Models

2.4.1 Network Morphogenesis

During simulation, the network undergoes radical changes in its structure. The morphology of SORN [23] model goes through two phases, decay and stable (*Figure 2. 2*). During decay phase, the number of connections between neurons in the network decreases due to the constant removal of connections between uncorrelated neurons by STDP rule. But gradually, the effects of STDP is counteracted by homeostatic plasticity mechanisms which leads to stable phase.

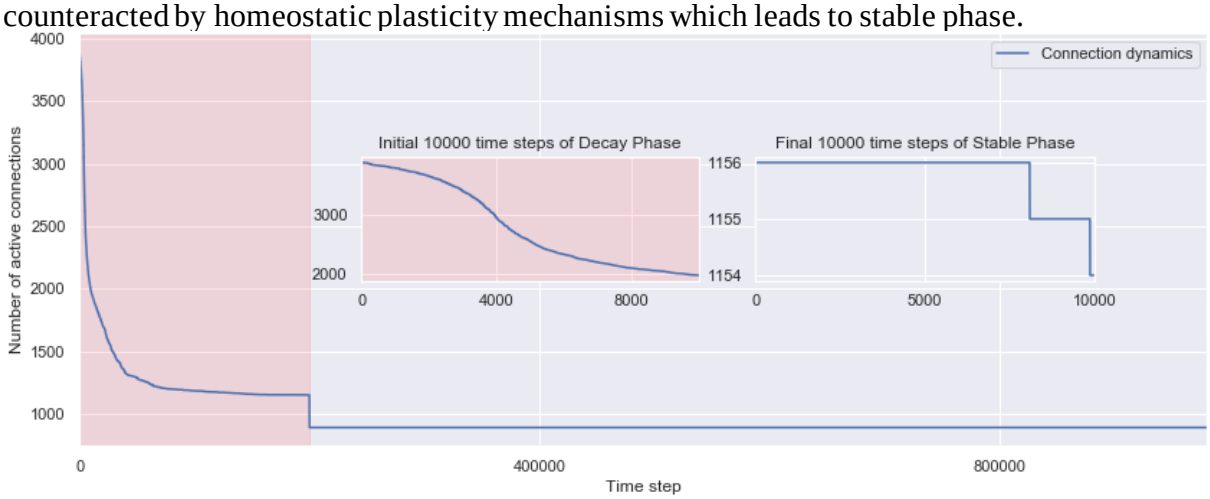


Figure 2. 2: Evolution of SORN [23] network connection during simulation

Figure 2. 3: Evolution of SORN [32] network connection during simulation

Similarly, in SORN [32] model, the structural plasticity (SP) rule add new connections between unconnected neurons, randomly at smaller rate. Once the effects of STDP is suppressed by intrinsic plasticity mechanisms, addition of new connections by SP and activity dependent strengthening of connections by STDP resulting in growth phase (Green shade) which indeed neutralized by IP later, which denotes convergence as shown in *Figure 2. 3*.

2.4.2 Synaptic Weight Distribution

As already discussed in section 2.1.2, the initial connections strengths are drawn (initialized) from either gaussian or uniform distribution. After 100000 time steps of network simulation, the histogram of connection strengths is measured and found that the weights are following positive or right skewed distribution as shown in *Figure 2. 4*. Such property of synaptic weights distribution obeys lognormal distribution of EPSPs in rat visual cortex [54] and proves the notion of skewed weight distribution observed in [55,56]. Further, the model converges to log normal distribution of synaptic efficacies reported in vivo [57-59] as shown in *Figure 2. 4E&F*.

2.4.3 Spike, Spike Train, Spike Rate and Inter-Spike Intervals (ISIs)

Mathematically, a spike (x) can be referred as an event occur at a time t_i and spike train as sequence of events ($x \in X$) occurred during time ($t_i \in T$) . The most commonly used distribution that abstractly describes the notion of neuronal spike trains at discrete time steps is Dirac delta function [73].

Let's assume the sequence of spikes generated by a neuron during the time window T as,

$$X = \{t_1, t_2, t_3, \dots t_N\} \qquad 2.12$$

where N is the number of spikes (Spike count) occurred during the time T.

Then, the Dirac delta function to describe an event or a spike at i^{th} time step is given by,

$$\delta(t) = 0 \;\; for\; t \neq i$$
$$\int_{-\infty}^{\infty} \delta(t)dt = 1 \;\; otherwise \qquad 2.13$$

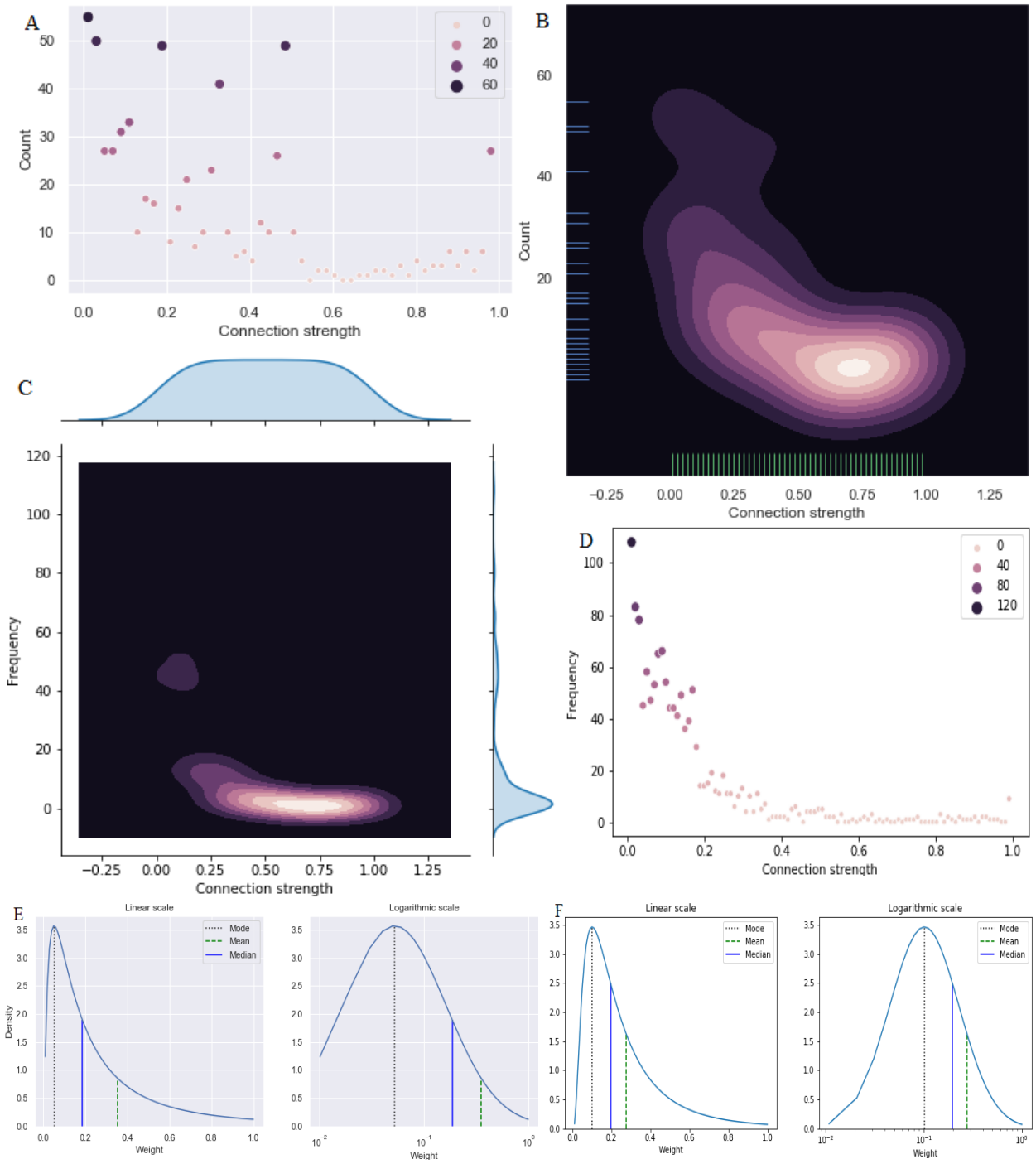


Figure 2. 4: Synaptic weight distribution at $5x10^4$ and $1x10^5$ time steps during simulation:

A) Scatter plot shows the distribution of connection strengths measured using histogram with bin size 100 at $5x10^4$ and B) it's density plot. C) Scatter plot of weight distribution at $1x10^5$ and D) its density plot E & F) Linear fit and lognormal fit of connection strengths at $5x10^4$ and $1x10^5$ time steps respectively.

The single spike (event) occur at time t_1 is then graphically represented as in *Figure 2. 5a* below,

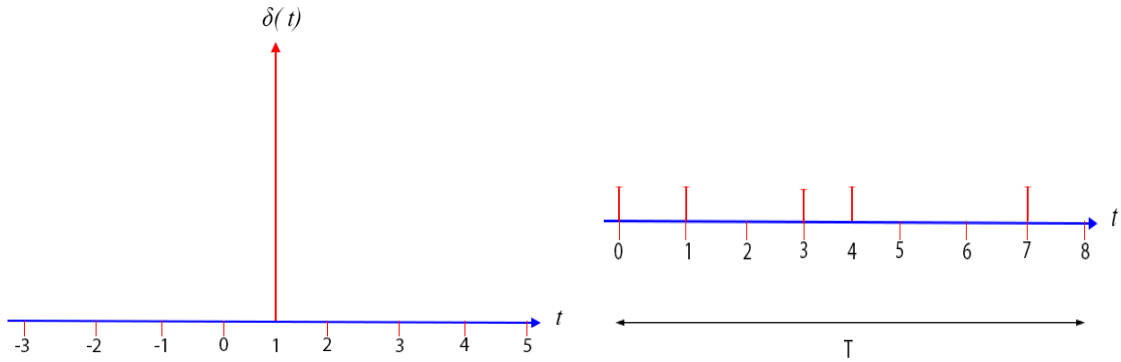


Figure 2. 5: ***a****: The δ-function of spike at time* t_1. ***b****: Spike train*

Similarly, the spike train consisting of N spikes, each occurring at times t_i during the interval T (*Figure 2. 5b*) can be represented by the sum of delta functions shifted by t_i.

$$\rho(t) = \sum_{i=1}^{N} \delta(t - t_i) \qquad 2.14$$

Using the definition of spike trains, spike rate or firing rate r(T) of neuron is given by the average of responses of the neuron over the time interval T.

$$r(T) = \frac{N}{T} = \frac{1}{T}\int_0^T \rho(\tau)\, d\tau \qquad 2.15$$

where τ is the time constant.

Since firing of neurons are non-deterministic [74], the mean firing rate of neurons over trials is considered as optimal measure, estimated by dividing mean number of spikes by the length of time.

$$\langle r(T)\rangle = \frac{\langle N\rangle}{T} = \frac{1}{T}\int_0^T r(t)dt \qquad 2.16$$

where $\langle r\rangle$ denotes the average of expected firing rate values.

Therefore, the spike rate of network with U number of neurons is given by,

$$R(T) = \frac{1}{U}\sum_{u=0}^{U} \langle r_u(T)\rangle \qquad 2.17$$

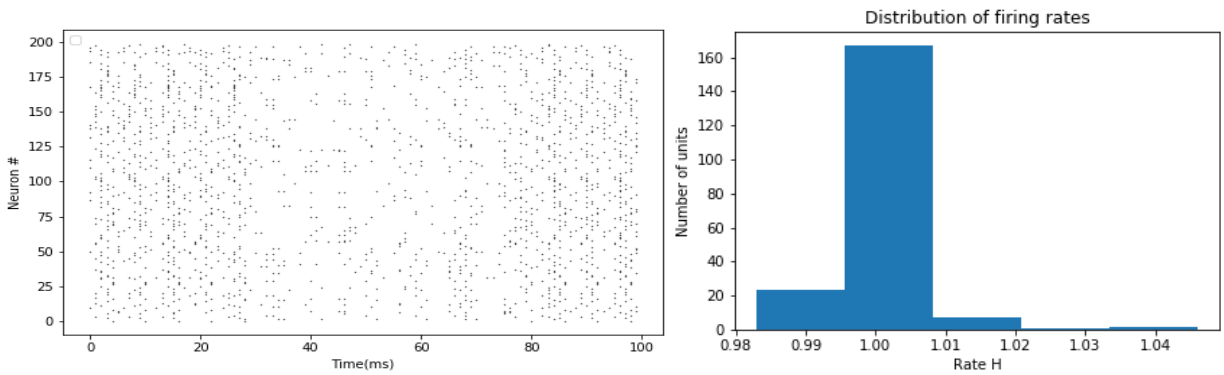


Figure 2. 6: ***a:*** *Scatter plot of spikes/spike trains of 200 neurons of SORN during the time window of size 100 and the line plot shows the firing rate/spike rate of the network of 200 neurons.* ***b:*** *Histogram of firing rate of the network with bin size 10.*

During simulation phase, the activity of SORN network driven by random inputs with gaussian noise are recorded and the spike rate of the entire network is measured with the time window size of 10. Results (Figure 2. 6) shows that, neurons produce spikes at the desirable firing rate, H which is 0.1.

The inter-spike intervals or spike time intervals is the time between adjacent spikes $(i_1, i_2, i_3, \dots i_k)$ occurring at $(t_1, t_2, t_3, \dots t_N)$ where $k = N - 1$.

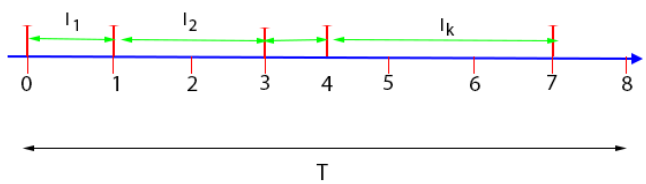


Figure 2. 7: Inter spike interval (I) of a single neuron during the time duration T.

Several studies showed that the statistical analysis on ISIs reveals the spatio-temporal firing patterns of neurons to its inputs and their integrating properties in the network [60]. Further, studies also proved that very short intervals (3msec or less) between spikes is necessary for neurons to decode the external stimulus related information [61]. Studies also reported that the ISI durations are exponentially distributed. Interestingly, the ISIs observed from any neuron in SORN model during simulation phase also follows exponential distribution as shown in Figure 2. 8.

This hypothesize that, this network of neurons driven by plasticity mechanisms could encode-decode the spatio temporal patterns from external inputs by efficiently transmitting the information between neurons within short period of time.

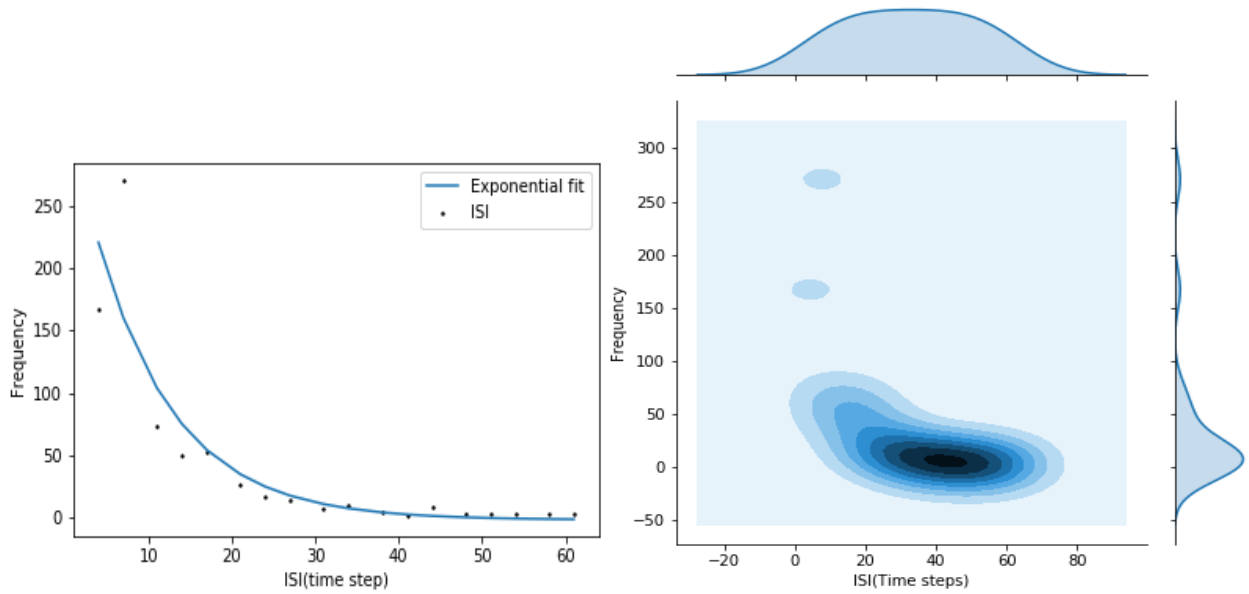


Figure 2. 8: ***a****: Distribution of ISIs with exponential fit of a random neuron in SORN;* ***b:*** *Density plot showing the higher ISIs of the chosen neuron appeared less frequently*

2.4.4 Correlation between Neurons

To study the correlation between each neuron in the network, spike trains are collected during simulation steps (998000 – 100000). The average spike correlation between each neuron in the network is measured using Pearson correlation. For an array with U neurons as variables with t observations, the correlation coefficient C between any two neurons is given by,

$$C_{uv} = \frac{\sum_{i=1}^{t}((u_i - \bar{u})(v_i - \bar{v}))}{\sqrt{\sum_{i=1}^{t}((u_i - \bar{u})^2 \sum_{i=1}^{t}(v_i - \bar{v})^2)}} \qquad 2.18$$

where u and v are the neurons of our interest; $\bar{u}$ and $\bar{v}$ are the mean number of spikes or spike rate. Therefore, the linear relationships between U neurons are obtained as an array of shape UxU using which, the average correlation is measured (Eq: 2.19).

$$C_{avg} = \frac{\sum_{u=1}^{U}\sum_{v=1}^{U} C_{uv}}{U^2} \qquad 2.19$$

In SORN, the average correlation coefficient remains close to 0 (Figure 2. 9), showing that there is not certain linear relationship between neurons generating spikes. Further, it also proves that each neuron driven by random stimuli (and noises in case of SORN [32] model) behaves independently even though their intrinsic behaviors like spike threshold, connectivity and connections weights are influenced by synchronized activity dependent plasticity rules.

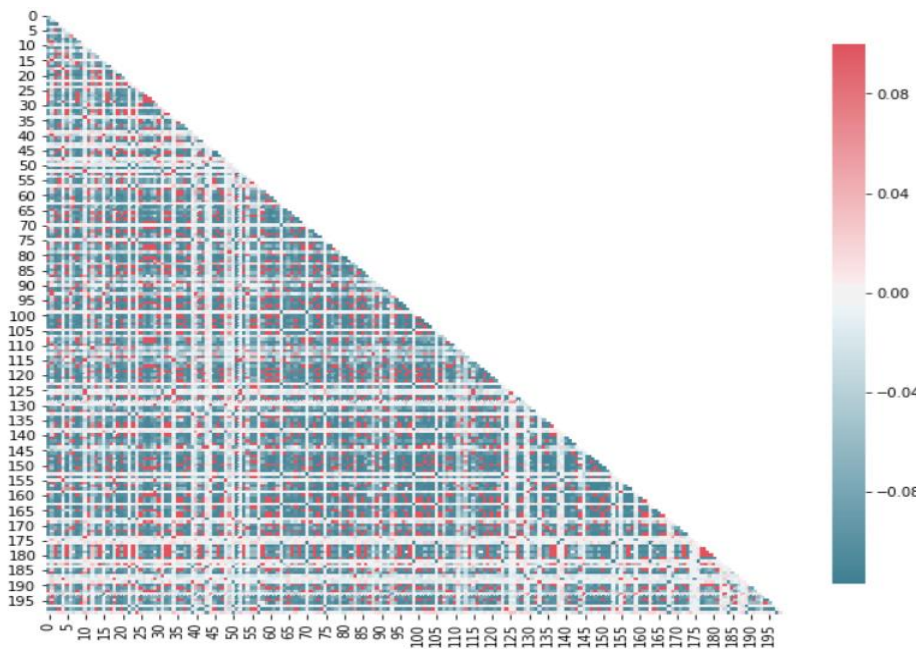


Figure 2. 9: Correlation coefficient between neurons

Studies[85,86] have shown that the above linear relationship between neurons measured by their activity over certain period (temporal scale) are also corresponds to their spatial relationship (spatial correlation) in the network. That is, the neurons that have higher temporal correlation are closer to each other. Therefore, investigating the connectome of this network (Figure 2. 10) can provide the signatures of functional properties between neurons in a single cluster or between clusters with respect to the external stimuli as observed in neocortex [87].

2.4.5 Poisson-like Network Activity

In neocortex, the time between successive spikes are found to be highly irregular and independent [62,63]. Such generation of spike, independent to all other spikes is referred as independent spike coding [64,65]. If any spike train obeys the above hypothesis, then it is said to follow Poisson process. To confirm biological plausibility of SORN models, the independent spike coding hypothesis is tested using Fano factor [66].

From equation 2.15, the mean spike count of neuron over the time length T denoted as $r(T)$. For K trials, the mean firing rate for k^{th} trial can be written as $r_k(T)$ while the mean of firing rates from all trials as $r_K(T)$. The deviation of firing rates of each trial from overall mean is given by,

$$\Delta r_k(T) = r_k(T) - r_K(T) \qquad 2.20$$

The variance of mean spike counts across K trials is the square of deviation (Eq. 2.20). By definition [75], Fano factor (F) is the measure of variability of spike counts observed in a time window and is estimated by the ratio of variance to its mean spike count.

$$F = \frac{(\Delta r_k(T))^2}{r_K(T)} \qquad 2.21$$

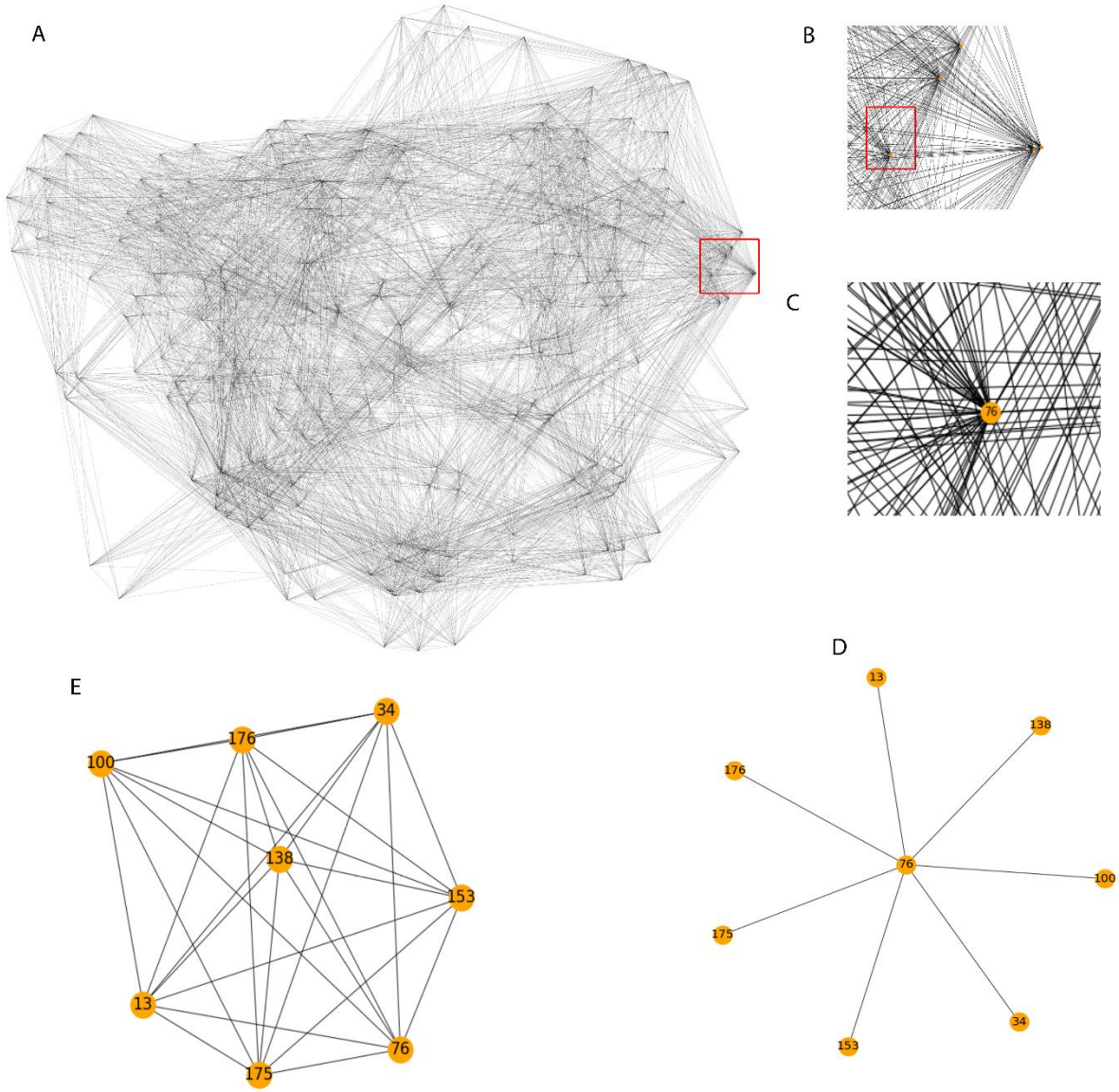


Figure 2. 10: Functional connectome of SORN: A: Connectome of entire network, B: 200% focus on cluster of neurons, C: 225% focus of a neuron in the cluster, D: Connectivity of the neuron with its highly correlated post -synaptic neighbors and E: The corresponding cluster structure

In SORN, since the mean firing rate is identical for neurons (*Figure 2. 6*) across different trials, the Fano factor measures (>0.8) from each neuronal activity (*Figure 2.10:Left*) proves that the spike generation in the network is a homogenous Poisson process [76].

However, studies [88] also confirmed that Poisson-like neuronal activity is not a universal feature of neocortex and speculated that spiking dynamics varies across cortical regions.

2.4.6 Spike Source Entropy

Spike source entropy is the measure of uncertainty about finding the origin of spikes in the network. It is defined as,

$$SSE = \frac{\sum_{i=1}^{N} p_i log_2 p_i}{log_2(1/N)} \qquad 2.22$$

Where,

$$p_i = \frac{Spike\ count\ of\ neuron\ i}{Spike\ count\ of\ entire\ network} = \frac{\rho_i(T)}{R(T) * N * T)}$$

is the probability that the spike is generated from the ith neuron, while T is the total time steps considered;

In SORN, the SSE is measured at constant time intervals (T = 20000) during simulation and found that the value stays close to 1.0 *Figure 2. 11* which confirms that the neurons are firing at identical rate at varying time intervals.

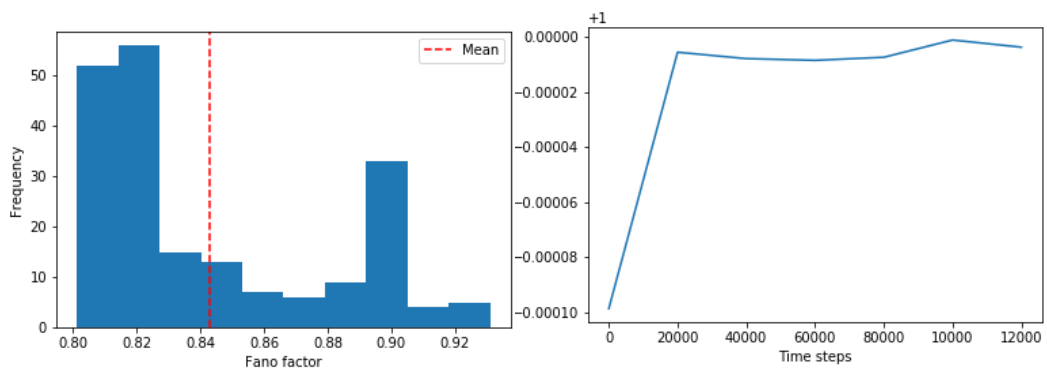


*Figure 2. 11: **a**: Histogram of Fano factor measure of neurons. **b**: Spike Source entropy*

2.5 Role of Plasticity Mechanisms in Network Dynamics and Organization

Biologically, spike timing dependent plasticity (STDP) provides correlation based Hebbian learning by strengthening and weakening the homosynaptic units(excitatory) in neocortex, whereas inhibitory STDP (iSTDP) introducing anti-Hebbian rules between excitatory and inhibitory neurons (heterosynaptic) [35]. Synaptic scaling (SS) adjusts the incoming excitatory synaptic strengths that impedes unconstrained potentiation (LTP) induced by highly correlated pre and post synaptic activations [51,52] following STDP. Another homeostatic mechanism, intrinsic plasticity is responsible for maintaining the firing rates of neurons [53] thereby balancing the overall activity of cortical circuit.

To study the significance of each plasticity mechanisms in earlier version of SORN [23], the network ($N^E = 200, N^U = 10, T_E^{max} = 0.5, T_I^{max} = 1.0, \lambda^W = 10$) is simulated for 50000 timesteps. With all plasticity mechanisms active, the network produces smooth poison like activity *(Figure 2. 12A)* with each individual neuron firing approximately around desired firing rate, H *(Figure 2. 14)*. The distribution of number of spikes in the network is found to be unimodal *(Figure 2. 12)* and centered at lower activity level as absorbed from biological neural circuits [51].

Further, the recent version of SORN ($N^E = 200, N^U = 10, T_E^{max} = 1.0, T_I^{max} = 0.5, \lambda^W = 10$)) is simulated for 50000 timesteps in presence of external noise ($\mu = 0, \sigma^2 = 0.01$) to study the influence of external noise including individual plasticity rules.

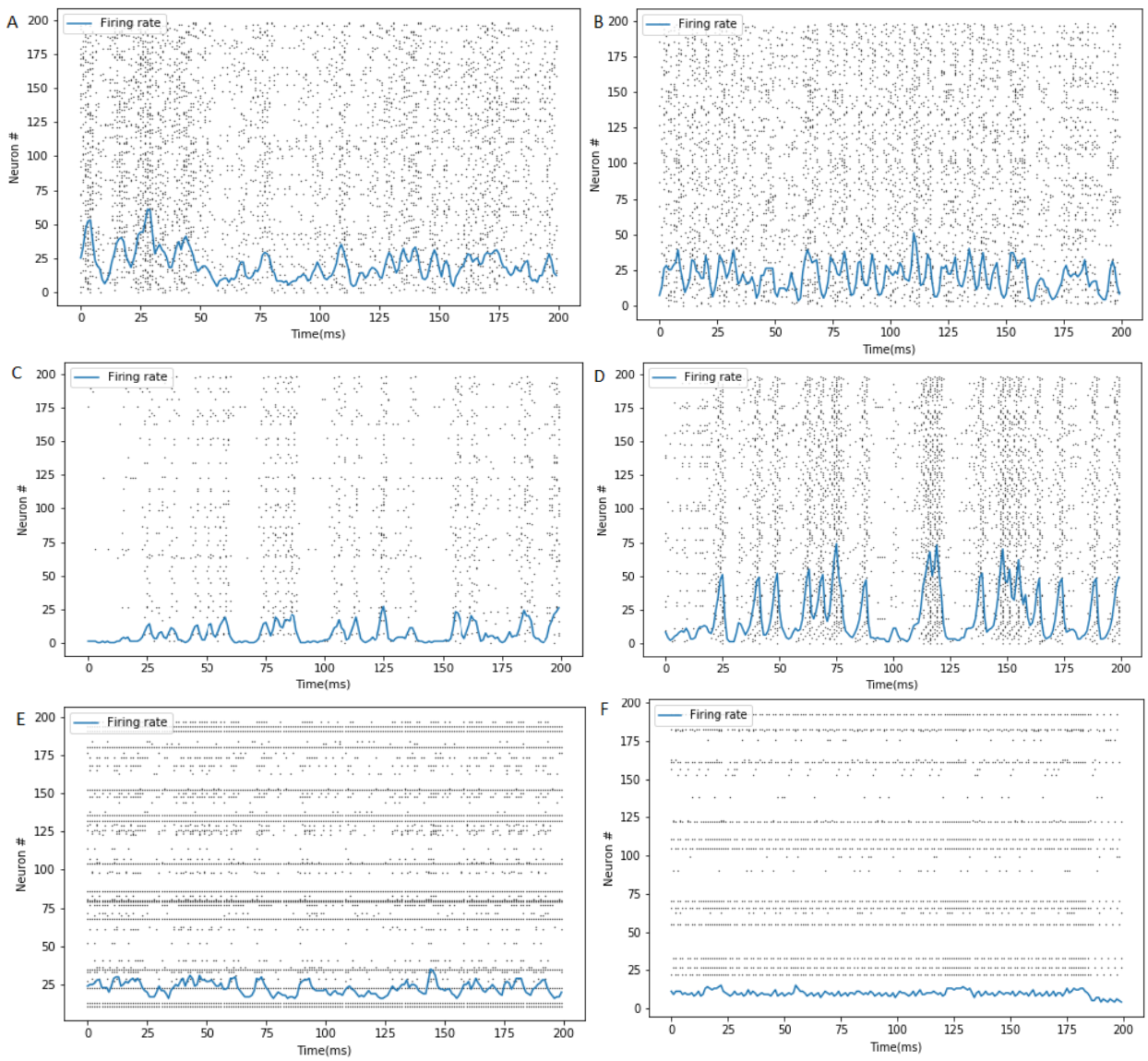


Figure 2. 12: Activity of SORN models observed after 50000 simulation time steps: A) Spike train of SORN [23] with all plasticity mechanisms B) Spike train of SORN [32] with all plasticity mechanisms. C) Spike train of SORN [23] missing SN D) Spike train of SORN [32] missing SN E) Spike train of SORN [23] missing IP F) Spike train of SORN missing IP; Note that the activity of network missing iSTDP is not shown since it is identical to the activity of SORN [23].

2.5.1 Contribution of Synaptic Scaling in Network Dynamics

In SORN [23], when SS is turned off, the network produces highly correlated synchronous burst of activity (*Figure 2. 12C*) and produces bimodal distribution of spikes per timestep (*Figure 2. 13*) even though each neuron firing approximately at target rate, H. This shows neurons in the network either remain silent or fire together at regular time intervals.

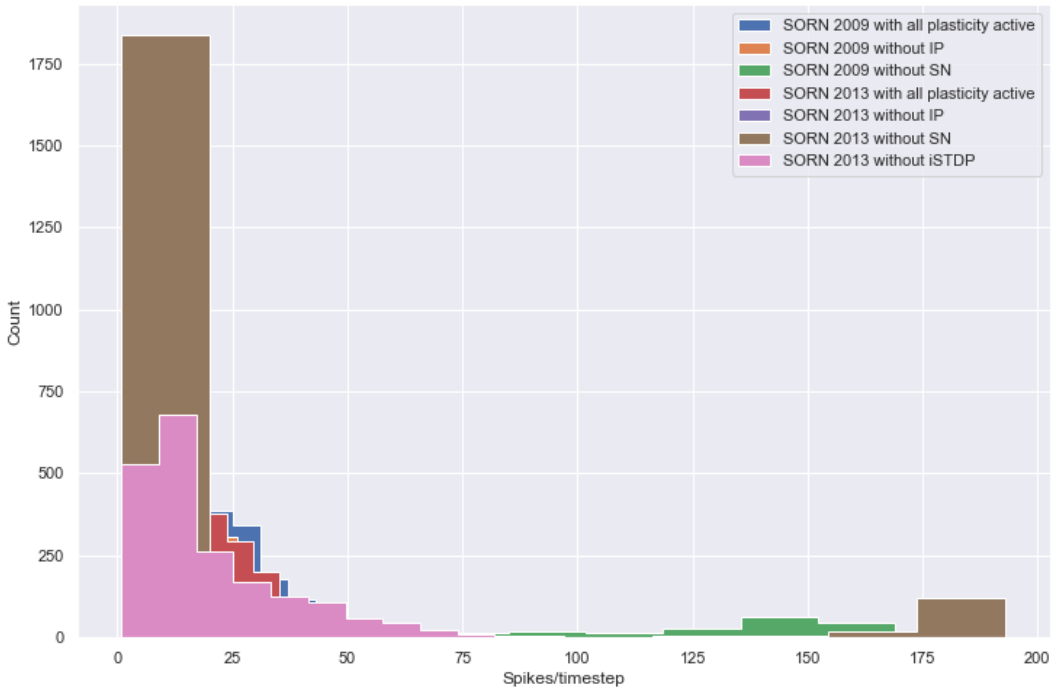


Figure 2. 13: Histogram of network firing rates of SORN [23] & SORN [32] at regular time intervals

Whereas in SORN [32], the absence of synaptic normalization produces abrupt increase in network firing rate *(Figure 2. 12D*). In both networks, SN has no influence over the individual neuronal firing rate (*Figure 2.14*). Therefore, SN is essential to preserve the network from elevated activity.

2.5.2 Contribution of Intrinsic Plasticity

In the absence of intrinsic plasticity, group of neurons develop epileptic behavior by being active synchronously while the other neurons remain silent (*Figure 2. 12 E&F*). The firing rate of individual neurons in the network is measured as the ratio of number of spikes per time window (size = 10). Histogram of average firing rates (*Figure 2. 14*) of each neuron in the network shows that few groups of neurons exhibits higher activity while the majority stay inactive. Hence, it is confirmed that intrinsic plasticity plays vital role in regulating the firing rates of each neuron by spreading the activity over entire network by forcing each neuron to contribute equally for the overall network activity.

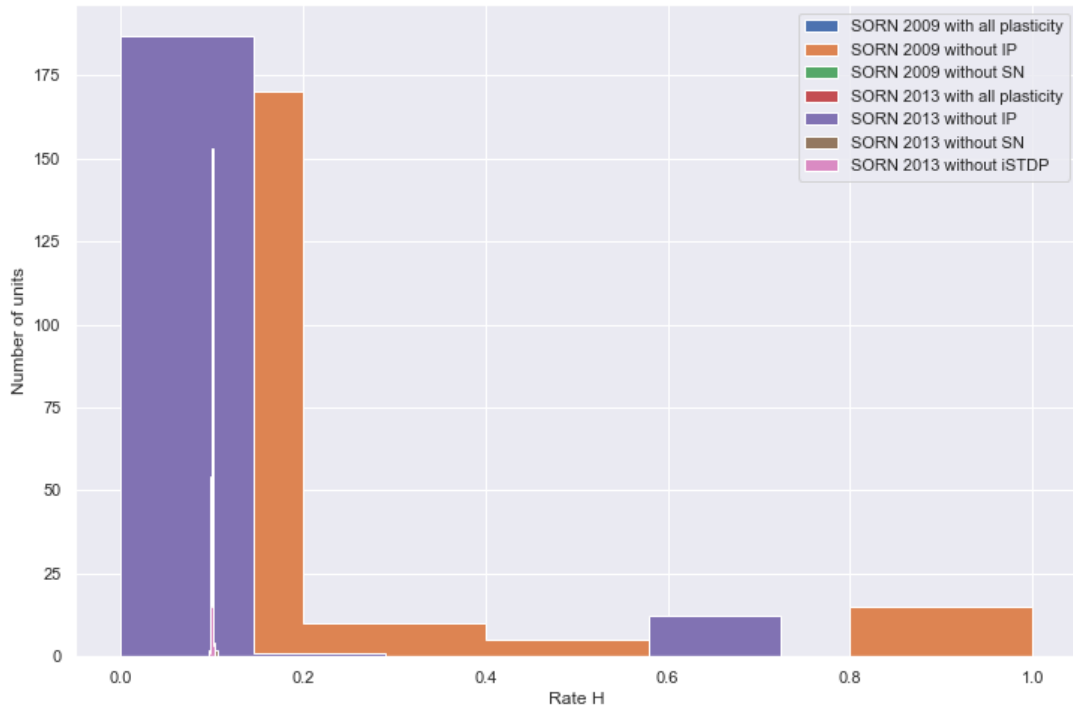


Figure 2. 14: Histogram of firing rates of neurons in SORN models at varying conditions

2.5.3 Contribution of iSTDP in Network Activity

It is hypothesized that both intrinsic plasticity and inhibitory STDP helps to keep the network at low firing rates.

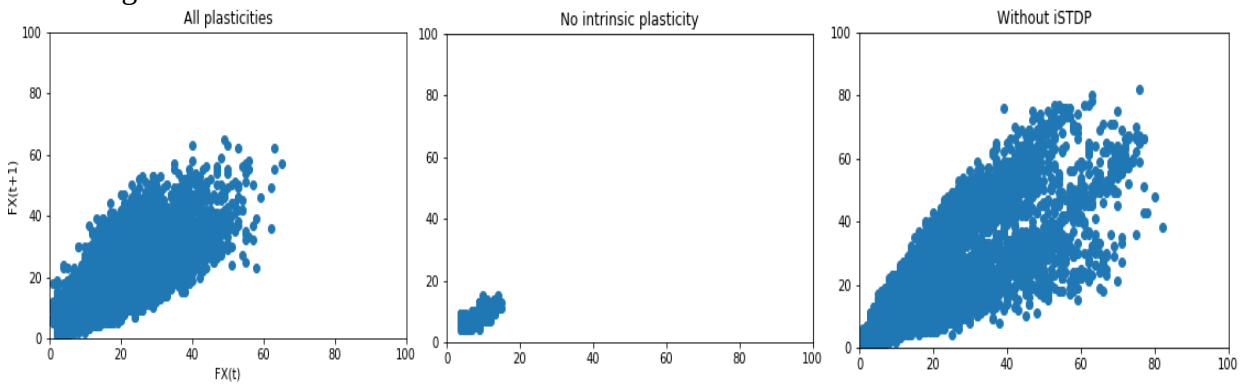


Figure 2. 15: Network activity of SORN [32] observed with all plasticity, without IP and iSTDP

SORN is simulated without iSTDP and observed the firing rates of the network (*Figure 2. 15*) and found that the networks without intrinsic plasticity produces very less activity. This is due to the fact that only few neurons are engaged in firing while the rest are inactive. In case of networks without iSTDP, burst of activity is observed and also the activity dies out at certain time intervals during simulation. Whereas the network with all plasticities ON, the activity never dies while the network also produces lesser activity compared to the network missing iSTDP. This shows that iSTDP and IP together regulates the firing rate of the network.

2.5.4 Role of Structural Plasticity in Network Reorganization

The positive feedback driven Hebbian STDP strengthen and weakens (followed by pruning) the connections between uncorrelated neurons during simulation period, which assumed to drift the network from low to highly correlated neuronal activity in SORN [23]. However, such influence of pruning connections in SORN [32] is compensated by structural plasticity. Interestingly, the networks without IP found to suppress the fast activity dependent pruning of connections between neurons which leads to drastic increase in number of connections between neurons in SORN [32].

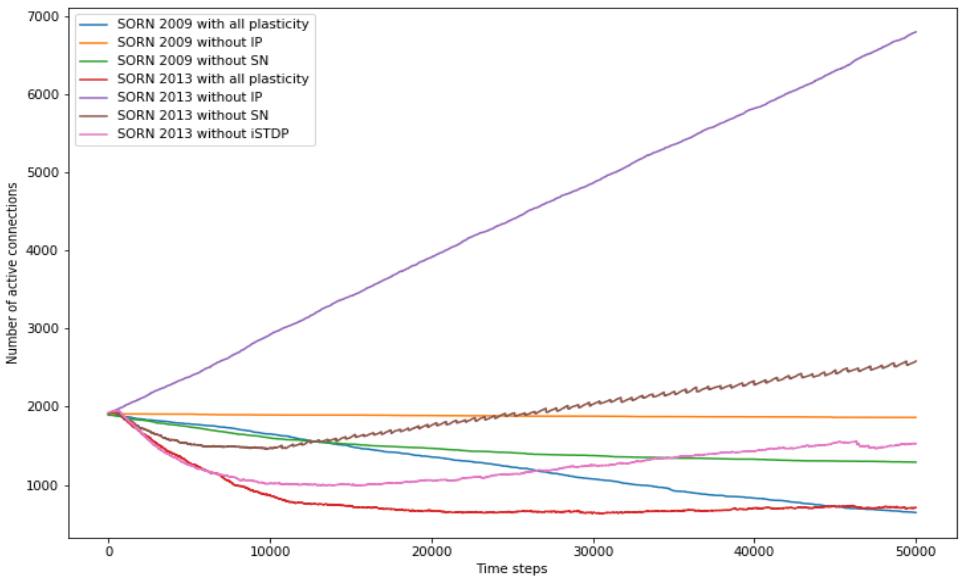


Figure 2. 16: Network connection dynamics during simulation period (0 – 50000 time steps)

Similar observations are found in networks missing SN, adding evidences to the hypothesis that SN and IP introduces negative feedback to counteract any instability in the network dynamics introduced by STDP.

2.6 Network Stability and Dynamics

Investigating the dynamics and stability of recurrently connected networks during simulation is critical to comprehend its behavior driven by external inputs. Mathematically, the evolution of state of the network at discrete time steps is given by,

$$x(t+1) = f\big(x(t), v(t+1)\big) \qquad 2.23$$

where $x(t)$ is the state of the network at t; v is the external input to the network. Therefore, behavior of the network over time is characterized by its recurrent activity and the external stimuli.

Consider SORN as a general dynamical system with temporal hidden units (the units whose states and connections undergoes radical changes over time as shown in *Figure 2. 2 & Figure 2. 12*). With varying input conditions, the network exhibits wide range of connection behaviors. For example, SORN [23] model with $N^E = 200$; $N^U = 15$; $T^E_{Max} = 0.75$; $T^I_{Max} = 1.0$ *and* $\lambda^W = 5$)

is simulated with random inputs from gaussian distribution ($\aleph(0,1)$) for 10^6 timesteps . The network achieved the

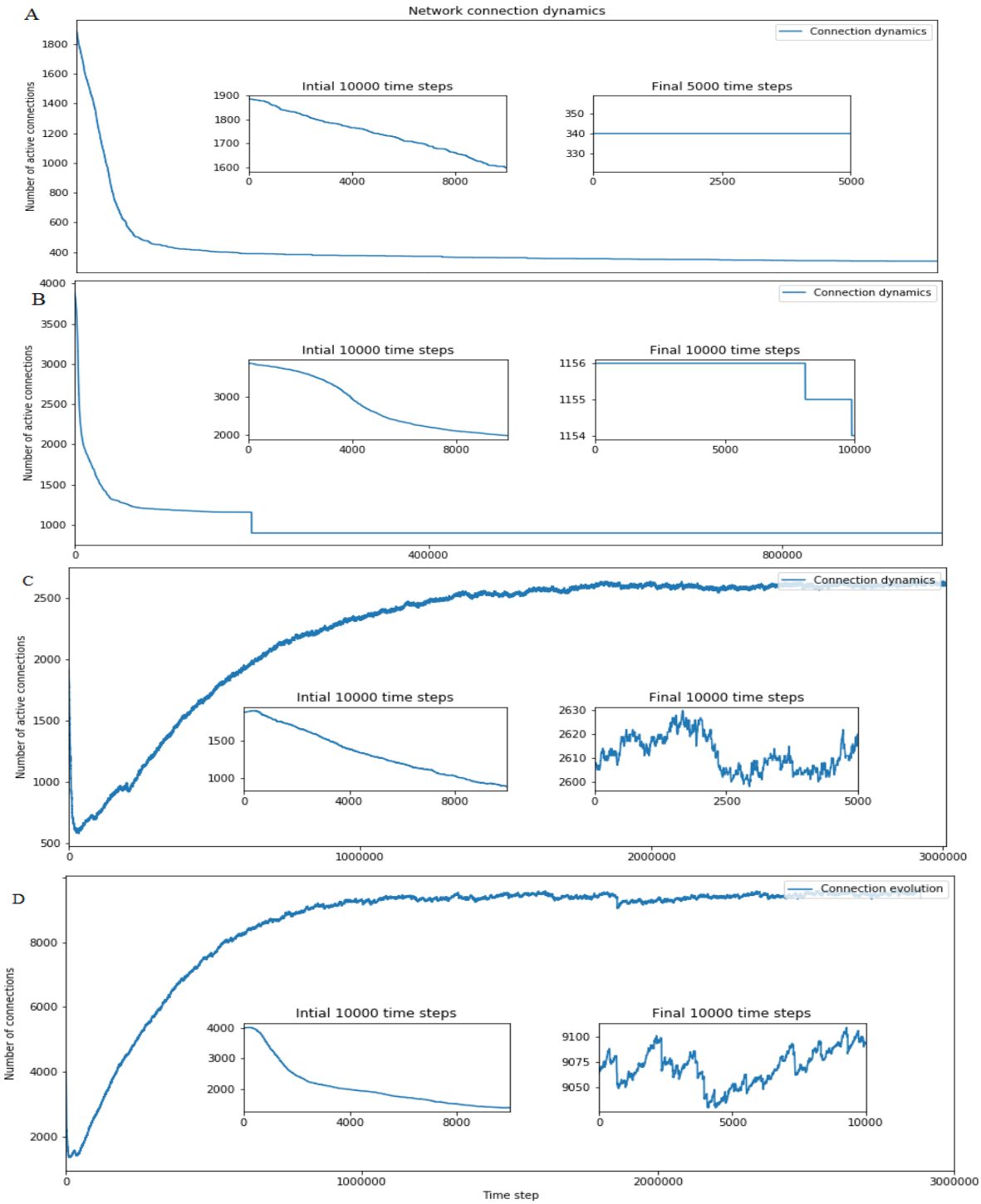


Figure 2. 17: Dynamics of SORN [23] model driven by A) Normally distributed random inputs and B) Random strong inputs C) SORN [32] model driven by normally distributed random inputs D) SORN [32] simulated under no inputs

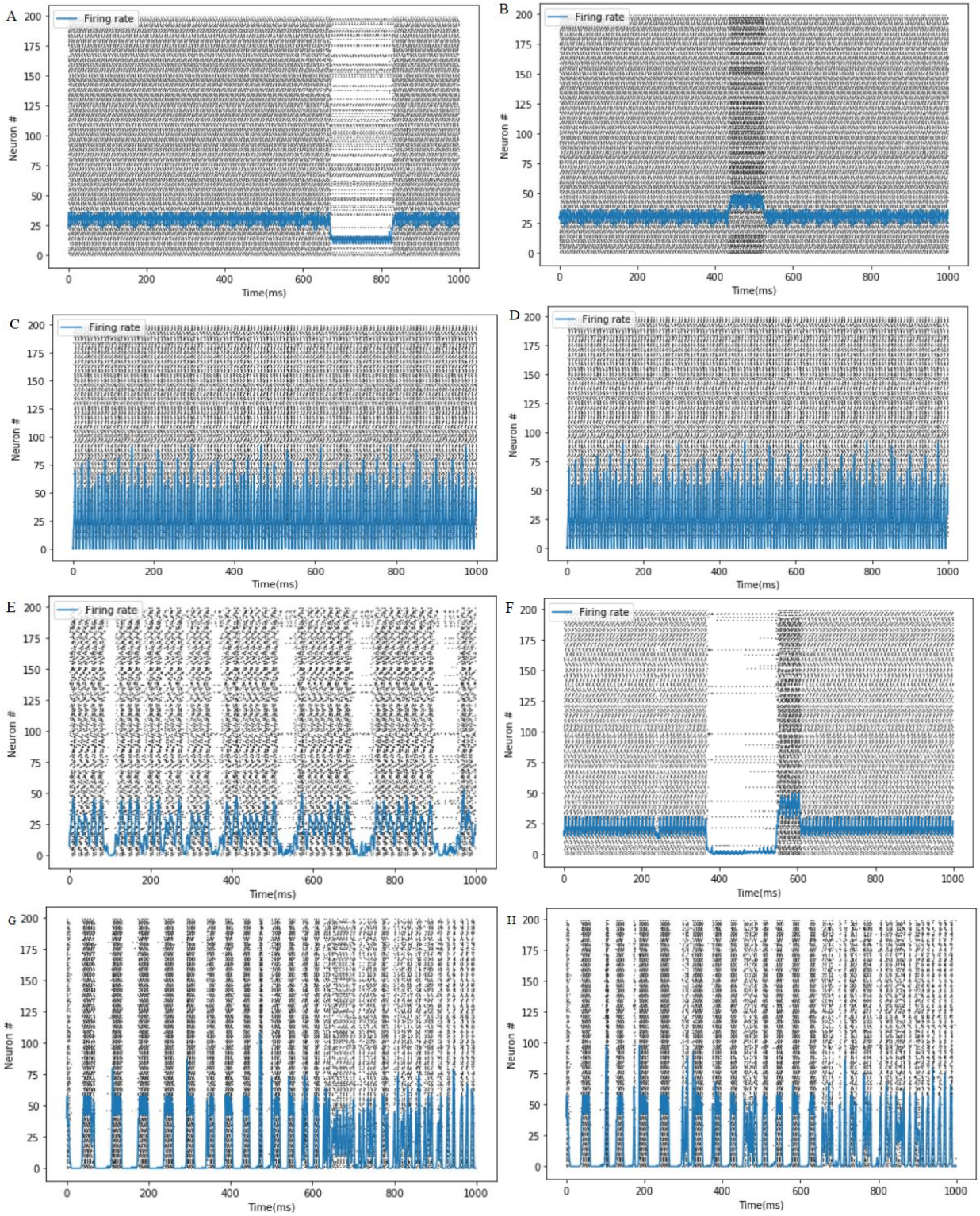


Figure 2. 18: Activity of SORN [23] network simulated under, A) Gaussian distributed inputs and its B) perturbed activity. C) Random strong inputs and its perturbed state activity D). E) Activity of SORN [32] network driven by Gaussian distributed inputs and its F) perturbed activity. G) Driven by no inputs and its H) perturbed activity.

stable phase at ~ 500,000 timesteps (*Figure 2.16*). With similar settings except ($\lambda^W = 10$) another network is simulated for 2x10^6 time steps with random inputs of strength (10^4) to make sure the input neuron fires at the instant of stimuli. The network progresses towards stable phase slower than its counterpart due to higher initial connection density as well as lower neuronal correlation induced by the random input to single neuron (*Figure 2. 17B*). Similarly, to investigate the influence of inputs, SORN [32] model $N^E = 200$; $N^U = 15$; $T^E_{Max} = 0.75$; $T^I_{Max} = 1.0$) with $\lambda^W = 5$, is simulated with gaussian inputs and without inputs ($\lambda^W = 10$) for 3x10^6. Due to structural plasticity and gaussian noise, both these networks reach stable phase with minor fluctuations in its connection (*Figure 2. 17 C&D*).

To study the stability of these networks, perturbation analysis is done using hamming distance by changing the state of a random neuron in each network and recorded the activity for next 10000 timesteps without external inputs.

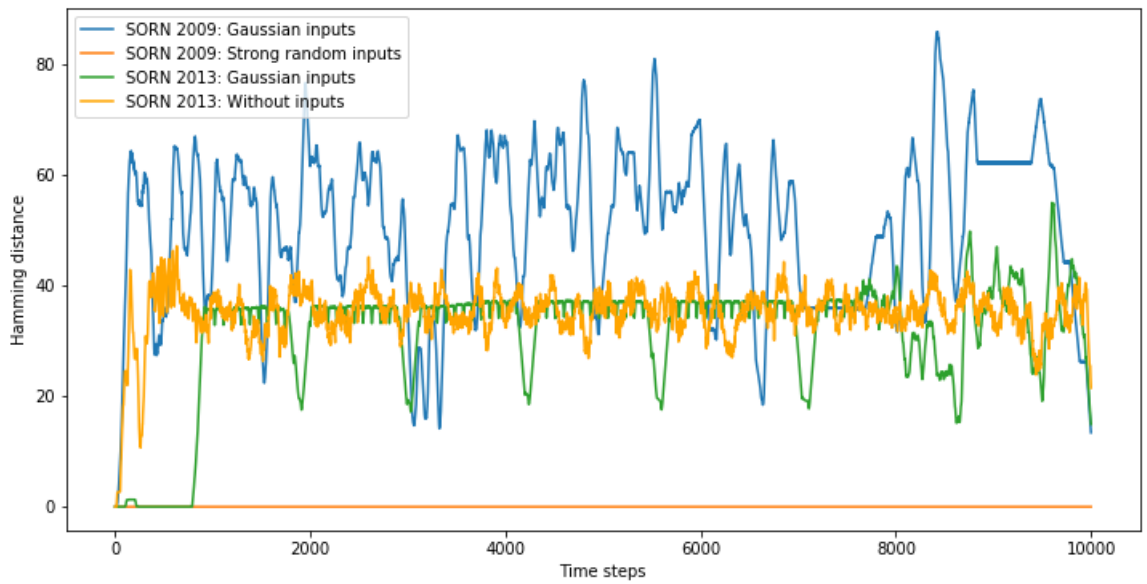


Figure 2. 19: Hamming distance measures

Hamming distance is measured by counting the number of neurons whose state in perturbed network are different from actual network state at respective time steps. For example, consider $x(t)$ as actual state of the network and $\tilde{x}(t)$ as perturbed network states. Initially at time step *t*, the hamming distance, $d(t) = 1$, since the activity of a random neuron is changed. The successive distance between these two versions of network activity is measured by resuming the simulation for 10000 timesteps. Hamming distance collected per time step were smoothened over steps of 100 and the plot (Figure 2. 19) shows that the network trained using strong random inputs has recovered from perturbation state (*Figure 2. 18C&D*) exposing the hidden self-correcting behavior. Whereas, the networks trained using gaussian distribution have diverged from stable to critical state and appeared to generate burst and low activity at regular time intervals (*Figure 2. 18A&B, Figure 2. 18E&F*). Further, it is observable that, SORN [32] network simulated with gaussian inputs has resisted the influence of perturbation for few time steps shows the significance of structural plasticity, iSTDP and the noise during simulation. However, the network trained without input has reached super critical state, proving the importance of inputs during simulation for network stability.

To summarize, while STDP and SS is explicitly responsible for learning and memory [24,26,27], combined action of all other plasticity mechanism helps to maintain the network in healthy dynamical state. Further, the network's self-organization through plasticity is necessary to achieve critical state but not needed to maintain critical signatures. Also, precise choice of noise and input level is crucial to maintain the network in critical regime [34].

CHAPTER 3: SORN FOR SEQUENTIAL DECISION-MAKING PROBLEMS

3.1 Concepts of Artificial Neural Networks

3.1.1 Perceptron

Artificial Neural Networks are a class of artificial learning systems that imitate the structural and functional aspects of human brain. Due to its ability to represent and learn complex linear and nonlinear functions, they achieve human or above human level performance in several data driven AI problems such as pattern, image, speech recognitions, classification tasks, etc.

Like network of neurons for brain, basic building block of neural networks are called Perceptron. Perceptron differs from the McCulloch-Pitts neuron model (seen in chapter 1) by the following factors,

i) Weights are not identical in the sense that they can be positive as well as negative.
ii) There is no inhibitory synapse
iii) Bias added to each input
iv) Neurons can take two states, where the output function y takes either [-1,1] or [0,1] based on the activation function.
v) Wide range of learning rules.

Using above descriptions, a simple perceptron model can be represented as in *Figure 3. 1*. The model consists of n number of input units (indexed as j) fully connected with the output neuron i, whose inputs are weighted by W_{ij}.

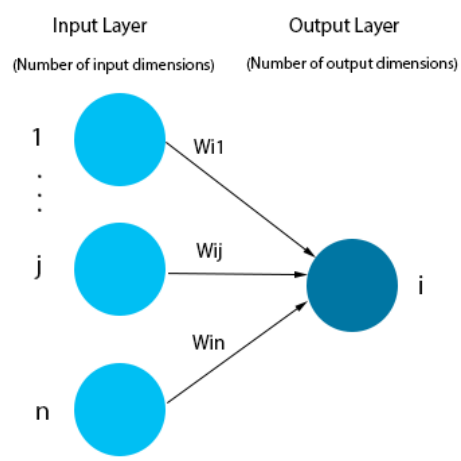


Figure 3. 1: A single layer Perceptron

The output of perceptron model is based on the type of activation function applied on unit i. That is, the activation function of unit defines the output of the unit given an input or set of inputs. Though there are number of activations functions under three distinct classes binary, linear and non-linear activation functions, there are only two types of activation functions (non-linear) employed for the experiments performed; Sigmoid and SoftMax.

3.1.1.1 Activation Functions

3.1.1.1.1 Sigmoid

Sigmoidal or logistic function is a type of non-linear activation function with characteristic S-shaped curve (*Figure 3. 2A*). In reference to perceptron model, sigmoid function maps the inputs (x_j) into output values (y_i) ranging between 0 and 1. If the input to unit *i* is given by,

$$z_i = \sum_{j=0}^{n} W_{ij} x_j + b_j \qquad 3.1$$

where *b* denotes the bias, then the output is measured as,

$$y_i = g(z_i) = \frac{1}{(1 + e^{-z_i})} \qquad 3.2$$

The bias value will be set to 0 in all the tasks for the ease of computation. Hence in further equations, bias term will be ignored.

3.1.1.1.2 SoftMax Function

SoftMax is the generalized form of sigmoid function (*Figure 3. 2B*), widely used in multiclass classification problems. It is used in the output layer of the network where the number of units (m) in that layer equals number of classes (should be greater than 1). The transfer function returns the probability of each class given in the problem. For example, the output of ith unit in the layer is measured by,

$$y_i = \frac{z_i}{\sum_{i=1}^{m} z_i)} \qquad 3.3$$

where, m is the number of units in the output layer.

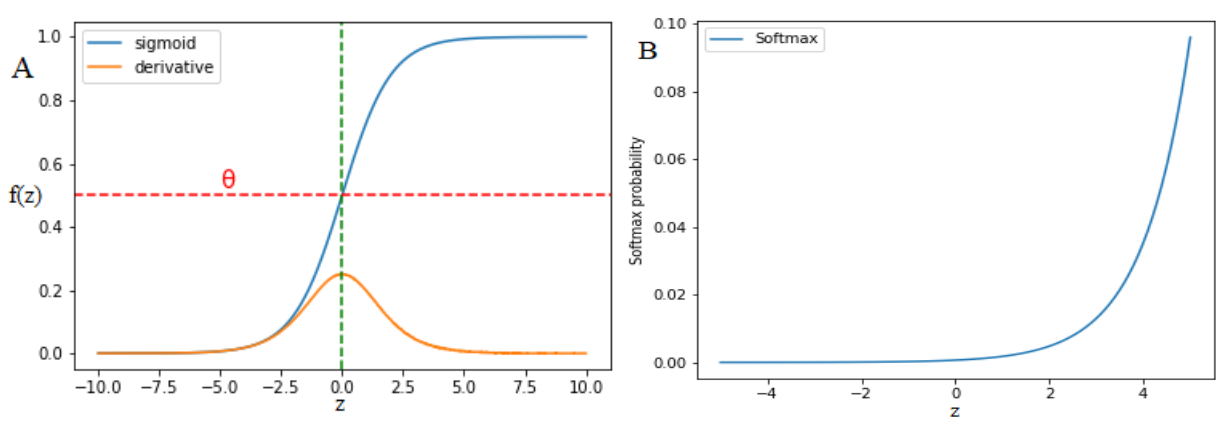


Figure 3. 2: Activation functions with their derivatives: A) Sigmoid B) SoftMax

3.1.2 Network Topologies

3.1.2.1 Feed Forward Neural Networks

Feed Forward neural networks (FFNNs) are the class of artificial neural network models in which the information (inputs) flows from input nodes *x* to output *f(x)* only in forward direction without any feedback i.e., output will not be fed back into the model. For any classification problems, given the set of input-target pairs (otherwise training set) as $\{(x^1, d^1), (x^2, d^2), \ldots (x^k, d^k), (x^K, d^K)\}$, the network has to map the inputs *(x)* to the target $d = f(x; W)$ by learning the values of the parameters W. That is, the aim of this network is to approximate the function $f(x; W)$. There are two types of FFNN models used for the tasks in this thesis, binary (for two classes) and multi class (for two or more classes) classification models.

3.1.2.1.1 Binary Class Model Representation

Consider the binary classification networks (*Figure 3. 3A*) with *n* input units connected to an output unit(s) through weights *W*. The output unit uses logistic or sigmoid function (Eq: 3.2) denoted by *g*. Then, the activation of output unit (y_1) given the parameters *(W)* and inputs *(x)* is given by,

$$y = f_W(x) = g\left(W_{11}x_1 + W_{ij}x_j + \cdots + W_{mn}x_n\right) \qquad 3.4$$

where, W_{ij} denotes connection weights from neuron *j* in input layer to neuron *i* in output layer. x_j denotes the jth input from input layer and *m* is the number of units in output layer; In this model *m=1*.

For naming convenience, the above equation is rewritten as,

$$y = f_W(x) = g(z)$$

where

$$z = Wx = (W_{11}x_1 + W_{12}x_2 + \cdots + W_{1n}x_n) \qquad 3.5$$

3.1.2.1.2 Multi Class Model Representation

Multi class model consists of n input units and 2 output units with an assumption that there are 2 classes in the target (*Figure 3. 3B*). The output layer is driven by SoftMax activation function (Eq: 3.3). Therefore, the activation of output units $(y_1 \text{ and } y_2)$ of the network with parameters *(W)* and inputs *(x)* is given by,

$$y_1 = g(W_{11}x_1 + W_{12}x_2 + \cdots + W_{1n}x_n) \qquad 3.6$$
$$y_2 = g(W_{21}x_1 + W_{22}x_2 + \cdots + W_{2n}x_n) \qquad 3.7$$

which can be rewritten as,

$$z_1 = Wx = (W_{11}x_1 + W_{12}x_2 + \cdots + W_{1n}x_n) \qquad 3.8$$

$$z_2 = Wx = (W_{21}x_1 + W_{22}x_2 + \cdots + W_{2n}x_n) \qquad 3.9$$

Then, the output, $y^k = f_W(x) \in \mathbb{R}^2$ i.e., output, $f_W(x^k) \approx d^k = \begin{bmatrix}1\\0\end{bmatrix}$ for class 1 or $f_W(x^k) \approx d^k = \begin{bmatrix}0\\1\end{bmatrix}$ for class 2, where d^k is the target output class for k^{th} input x^k.

Also note that, the binary class model can also perform the same task with output either 0 or 1.

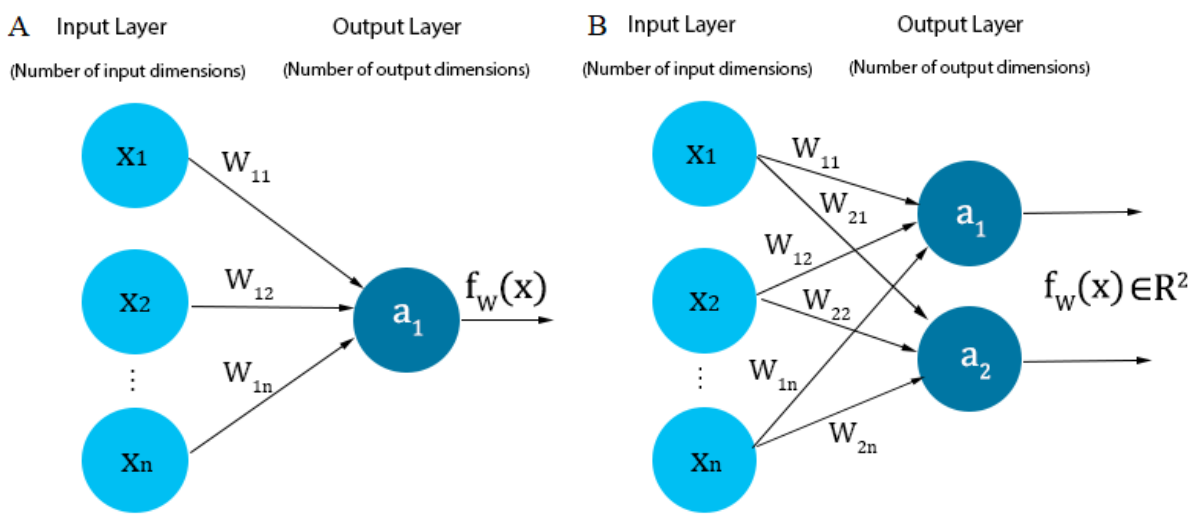


Figure 3. 3: Feed forward neural network model representations: A) Binary classification model B) Multi class classification model

3.1.3 Supervised Learning Algorithm

The prediction or classification performance of above discussed models are based on the type of learning algorithm. This section will introduce basic concepts of supervised learning algorithm.

Supervised learning algorithms are used to fit the parameters (weights) of FFNNs provided the target class for each sample in the training set. There are several methods to learn the parameters of the model, but I will focus on the approach that illustrates statistical learning i.e., minimizing the cost function.

3.1.3.1 Cost Function

Given the training set $\{(x^1, d^1), (x^2, d^2), \dots (x^k, d^k), (x^K, d^K)\}$, the error function is defined by,

$$\delta_i^k = y_i^k - d_i^k \qquad 3.10$$

where, y_i^k is the activation of ith neuron in the output layer, while d_i^k is the desired response for the input x^k. Then the instantaneous sum of square errors is given by,

$$\xi^k = \frac{\sum_{i=1}^{m} (\delta_i^k)^2}{2} \qquad 3.11$$

The cost function or error energy (otherwise the "gradient") is the average of squared error over K samples

$$\xi_{av} = \frac{1}{K}\sum_{k=1}^{K} \xi^{k} \qquad 3.12$$

This process of forwarding the inputs into the model until measuring the error is called forward propagation (*Figure 3. 4*).

Note that, sample size K in the above equation doesn't mean that the error is measured for the entire K samples. The training dataset can be sliced into chunks of samples called batches of size K.

3.1.3.2 Backpropagation

The target for the learning algorithm is to reduce the error values (Eq: 3.12) during forward propagation of samples from the training dataset by gradually tuning the parameters of the models. Backpropagation is the technique for tuning the weights by propagating the gradient back into the network. Such parameter optimization method is called gradient descent.

3.1.3.2.1 Gradient Descent Optimization

The aim of gradient descent algorithm is to find the minimum of the cost function by updating the parameters in the opposite direction of the gradient i.e. towards negative gradient. The degree at which the parameters are tuned at each step during training is determined by a constant value called learning rate α as shown in Eq: 3.13.

$$W = W - \alpha\, dW \qquad 3.13$$

where,

$$dW = \xi_{av}\, X^{T} \qquad 3.14$$

and X is nxK matrix.

The process of feeding inputs (forward propagation) and back propagating the error to optimize the weights is called an Epoch. Therefore, the above discussed methods are applied iteratively for T number of epochs during which, the model must be optimized to produce desirable performance with acceptable error.

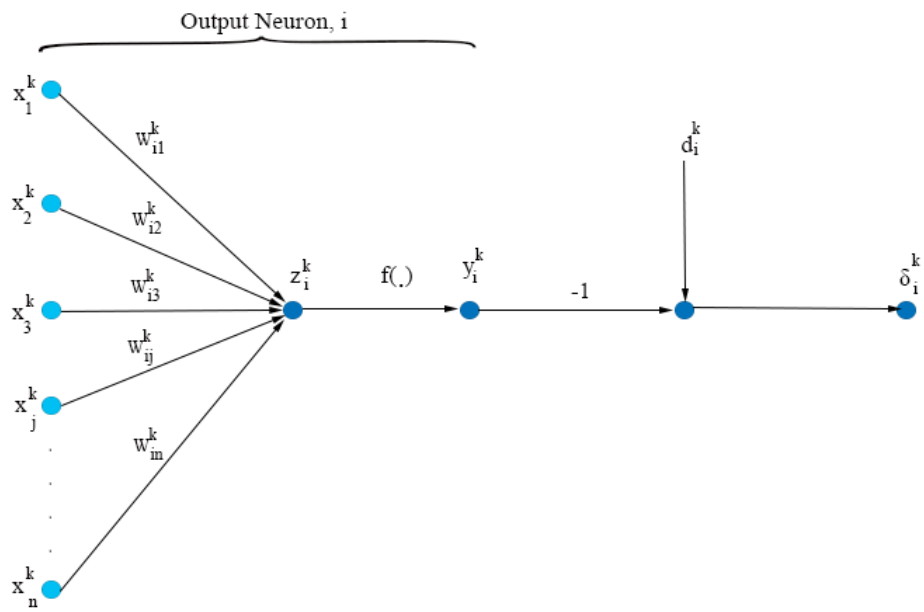


Figure 3. 4: Forward signal flow graph highlighting the details of output neuron i

3.1.3.2.2 Learning Rate

Consider, the gradient function as a convex function (*Figure 3. 5Left*). As discussed above, the target for the learning algorithm is to achieve the minimum (global minima) of that function. At each iteration of the algorithm, i.e. for each batch of training samples, the fixed learning rate α, decides the size of its step towards the point of global minima called convergence through Eq: 3.13.

An optimal learning rate always guarantees a near convergence in gradient descent [79], but it requires tuning in practice. If it is larger, the algorithm takes larger values (steps) while updating weights and it may end up at the sides instead of bottom of the error function (red lines in *Figure 3. 5:Left*). However, cost function for many real-world problems is not always convex. It could be non-convex (*Figure 3. 5: Right*). In that case, if the learning rate is not large enough it might strike in the local minima (Red circle). To avoid such instances, the learning rate must be tuned with respect to the error energy during training. One among, notable adaptive gradient descent technique is RMSProp.

3.1.3.2.3 RMS Prop Optimization

Root Mean Square Prop is an algorithm used to speed up the training by tuning the strength of weight updates at each iteration as follows,

Given dW from (Eq: 3.14),

The gradient in RMS prop at t^{th} epoch is measured by,

$$\nabla_{dW}^{t} = \beta\, \nabla_{dW}^{t-1} + (1-\beta)(dW^{t})^{2} \qquad 3.15$$

where β is momentum in other words the adaptive rate constant.

Then the weights are updated as,

$$W^{t+1} = W^{t} - \alpha \frac{dW^{t}}{\nabla_{dW}^{t}} \qquad 3.16$$

But, due to element-wise squaring operation on dW^{t}, during training ∇_{dW}^{t} may get closer to zero. Therefore, to avoid the weights to be updated with higher values, a small constant value ε is added and the Equation 3.16 can be rewritten as,

$$W^{t+1} \quad = W^{t} - \alpha \frac{dW^{t}}{\nabla_{dW}^{t} + \varepsilon} \qquad 3.17$$

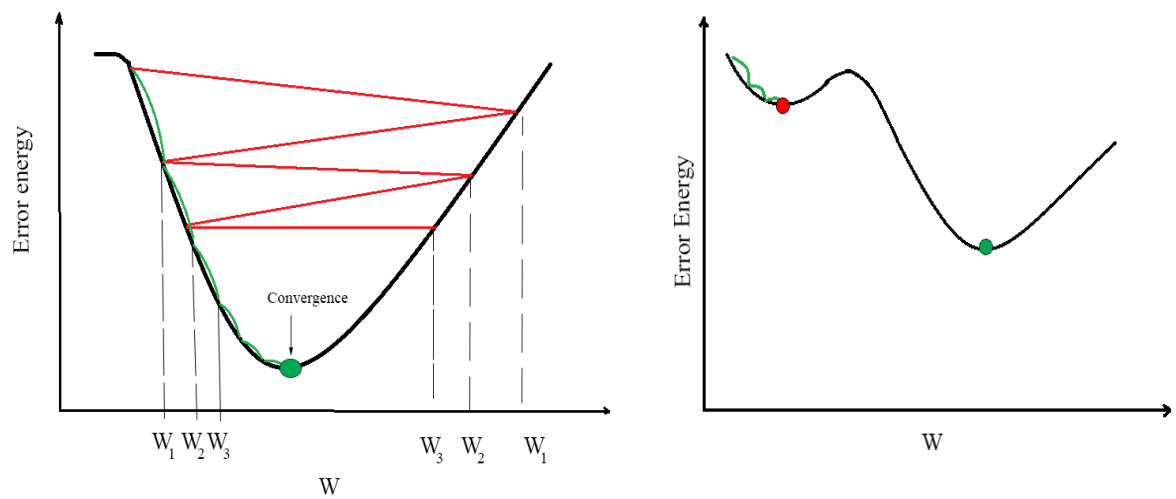


Figure 3. 5: Learning rate vs Cost function: Left: Optimal (Green) learning rate and Large learning rate (Red). Right: Fate of small learning rate

3.2 SORN for Sequence Prediction

Using the concepts of ANN, a time series prediction task is designed in order to test the ability of SORN learning the hidden features in input sequence and to study how it responds to each input with distinct dynamic states. The task is performed on SORN models which achieved stable phase and evaluate performance of each model that were simulated under varying input conditions:

i) $Model\ \alpha$: SORN [23], with 200 neurons simulated under strong random inputs
ii) $Model\ \beta$: SORN [23], with 200 neurons simulated under Gaussian random inputs
iii) $Model\ \gamma$: SORN [32], with 200 neurons simulated under Gaussian random inputs
iv) $Model\ \delta$: SORN [32], with 200 neurons simulated without inputs.

3.2.1 Training Data Generation

The training data is a sequence of numbers from 1 to 10, which are then converted into unique one hot vectors of size equal to the number of neurons in the reservoir i.e. a random input neuron in the reservoir is mapped to each input value in the sequence. Whereas in case of network which is simulated without any inputs, a neuron from the reservoir is chosen randomly for each class in input. The strength of the external input is strong $1e^4$ to make sure the input neuron fires once it is triggered.

3.2.2 Schematic Representation of Network Training

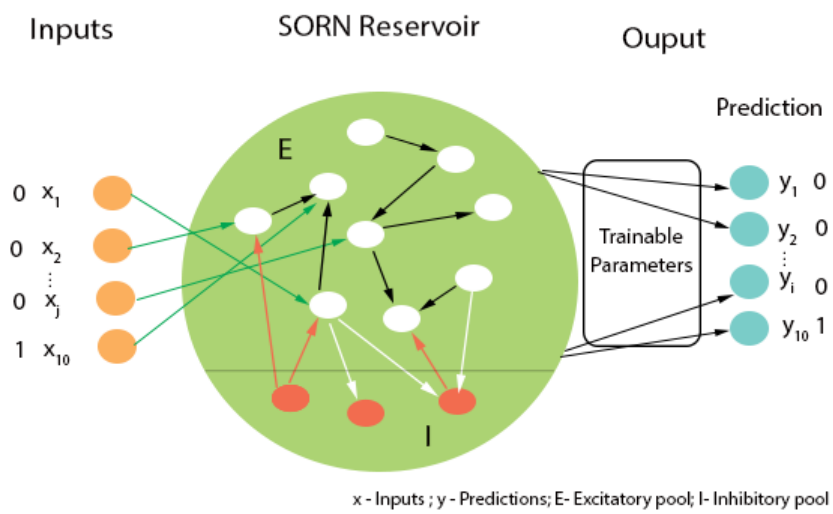


Figure 3. 6: Network training schema

The training involves two phases: Plasticity phase and Training phase. During plasticity phase, the inputs are fed into the network for certain time steps, allowing the network to learn the sequence in its inputs directed by morphological changes in its connection structure. While in training phase, all the connection weights of the reservoir are frozen and only the linear output layer (trainable parameters in *Figure 3. 6*) connected to the reservoir is optimized to predict the target sequences (for this task, it is input itself) using supervised learning algorithm supported by RMSProp with $\alpha = 0.001, \varepsilon = 1e^{-8}$ and $\beta = 0$. The hyperparameters of SORN network are fixed at ($N^E = 200, N^U = 10, T_E^{max} = 1.0, T_I^{max} = 0.5, \lambda^W = 10$). The duration of plasticity phase is set to 1000 timesteps (10 epochs) while the training phase has 20,000 timesteps (2000 epochs)

3.2.3 Results and analysis

To confirm, whether the network has learned the structure in its input during plasticity phase, the recurrent activity (Eq: 2.9 & Eq: 2.10 in Chapter 2) is used to predict the next number in the input sequence which is followed by prediction from network state influenced by external input (Eq: 2.7). From the results (*Figure 3.7 & 3.8*), the networks α and δ simulated using the strong inputs and without any inputs respectively dominates other versions of the network in performance. It is obvious that, the inputs for this prediction tasks are one hot vectors with strong input stimuli ($1e^4$) upon which the network α was simulated. Therefore, sampling both simulation and training data

from same or similar distribution is necessary to achieve acceptable performance from the network.

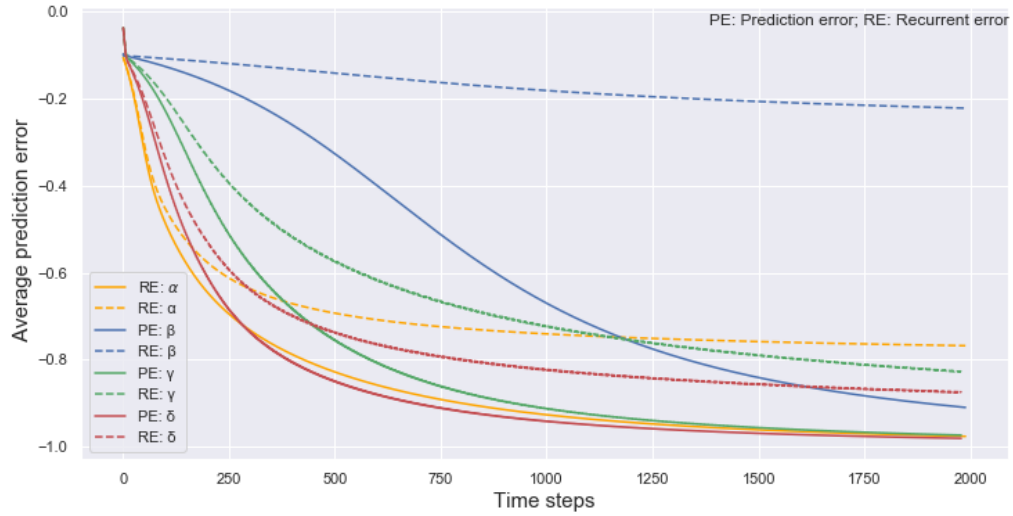


Figure 3.7: Average errors from network state driven and recurrent driven predictions from models of SORN

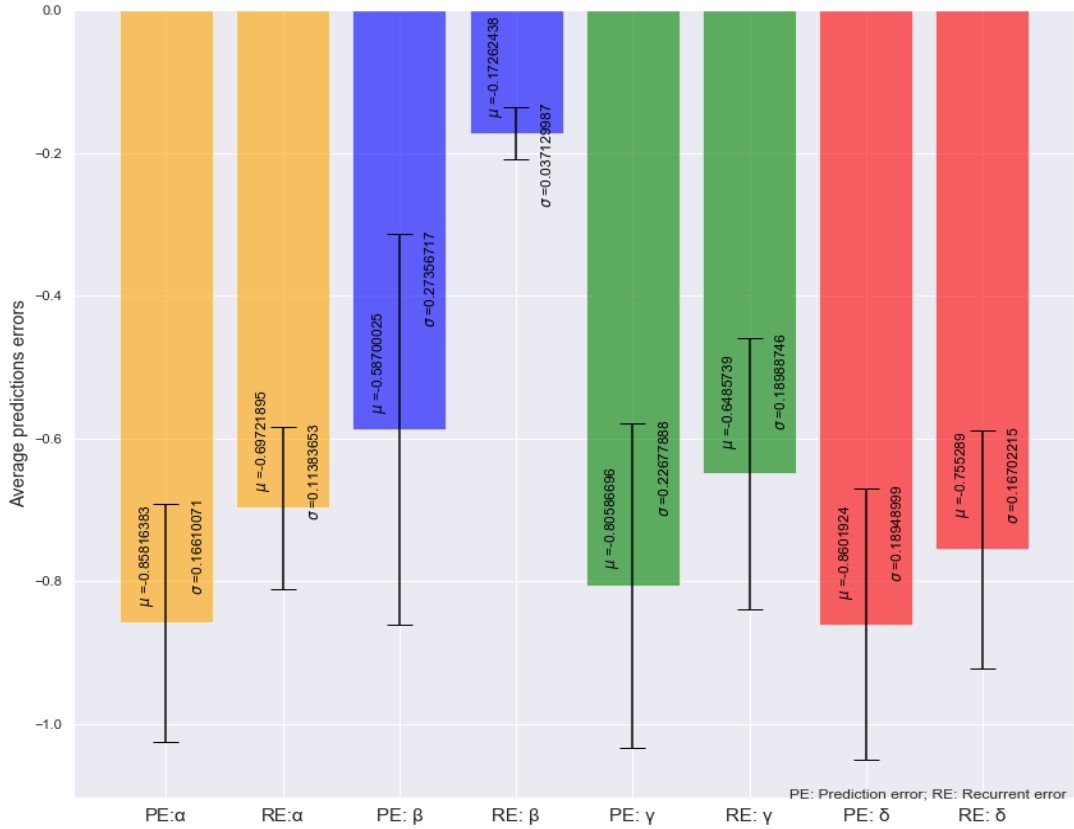


Figure 3. 8: Bar plot showing mean and standard deviation of average prediction errors from various versions of SORN

Further, the network simulated without any input δ also produced competitive performance, which shows that, it is also reasonable to simulate the network without any input if the distribution of training data is unknown. It is noticeable that, during criticality analysis (*Figure 2. 17 & Figure 2.*

18) this network has failed to recover from the perturbation, adding proof to the speculation made by authors of SORN [23] that criticality shouldn't be considered as performance factor during prediction tasks, since the network can incorporate the temporal information in external stimuli into its dynamics. Adding more evidence to the above speculation, the network SORN [32] simulated under gaussian inputs has produced better prediction performance than other network, although the prediction from its recurrent activity shows the network has failed to capture the temporal information into its dynamics (See S1 for details).

From these sequence prediction tasks, it is evident that, SORN models which are simulated using inputs that closely approximate the distribution of values in the training data, can optimize itself during plasticity phase and generate predictable network dynamics during training phase. Results also proves that the SORN [32] network have responded reliably than SORN [23] (higher fluctuations in performance) to different external stimuli (Figures S1.) strongly support the claim of authors about the criticality of noise [34] in SORN.

3.3 Reinforcement learning

Reinforcement learning (RL) involves an agent with finite set of actions and a dynamic environment with finite set of observations or states (Figure 3. 9). Unlike supervised learning algorithms, there is no target signals but reward signals which indicates how well the agent is doing in the environment at each state. Hence, the goal for the agent is to learn the optimal behavior to decide which action to perform at a state to maximize the cumulative rewards in subsequent time steps.
Any RL problem can be inspected by three observable variables,

i) State space S: Set of states, $s \in S$
ii) Action space A: Set of actions, $a \in A$
iii) Reward: A scalar value $r \in R$, received for transition from state s to s' by taking the action a

The transition $T(s, s')$ by taking the action a is decided by a function called Policy $\pi, \pi(.): S \rightarrow A$. That is at each state, the policy informs the agent which action to perform. The processes of finding the policy (optimal behavior) at any state is based on Markov Decision Process (MDP).

3.3.1 Markov Decision Process

Given the current state S_t with action performed A_t and the reward R_t, the probability of moving into new state S_{t+1} and receive reward R_{t+1} is defined by MDP as,

$$p(s'|r, s, a) = \mathrm{P}[S_{t+1} = s', R_{t+1} = r | S_t = s, A_t = a] \;\; \forall s, s' \in S, a \in A \qquad 3.18$$

The above equation represents first order Markov, since the future step is depending only on the present and not on the past steps. Further, the equation characterizes environment dynamics using state transition probability p. As discussed, the cumulative future rewards the agent receive for being in state s' will be measured by,

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots \qquad 3.19$$

where $\gamma \in (0,1)$ is called discount factor. If $\gamma = 0$, then the process only considers the immediate future reward and if $\gamma = 1$, all the future rewards will be considered. For convenience, the Eq: 3.19 can be reformulated as,

$$G_t = \sum_{t=1}^{T} \gamma^{t-1} R_{a_t}(S_t, S_{t+1}) \qquad 3.20$$

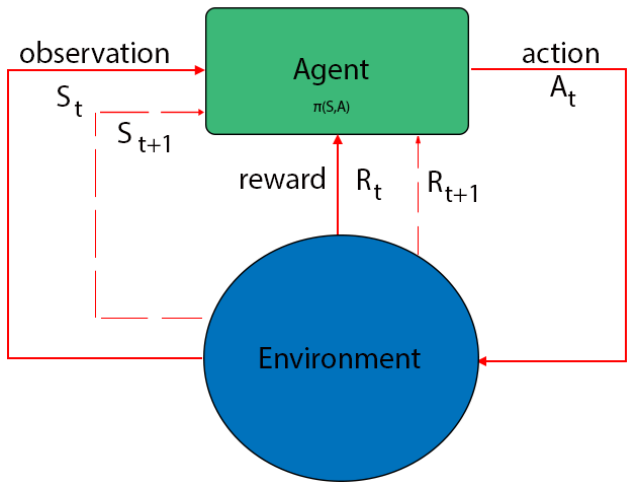


Figure 3. 9: Agent and Environment feedback loop in Reinforcement learning

and the policy with parameter W, that informed the agent to perform action a at state s to move to s' is,

$$\pi_W(s,a) = P(a|s,W)\ \forall\ a \in A, s \in S, W \in \mathbb{R}^l \ \ where\ l \ll |S| \qquad 3.21$$

3.3.2 Policy objective function and gradient

MDP generates a tuple (S, A, R, p, γ) with sequence of states, actions and rewards for finite time steps $(s_1, a_1, r_1 \dots s_t, a_t, r_t, s_T, a_T, r_T)$ is considered as an Episode, τ. The objective of RL is to increase the expected total reward G_τ by optimizing the parameter W.

$$J_\tau(W) = E_{\pi_W}[G_\tau] = \sum_{\tau} \pi_\tau(s, a; W) R_\tau \qquad 3.22$$

Therefore, given a policy objective function $J(W)$, policy gradient algorithm will search for the local maximum in $J(W)$ using gradient ascent with respect to parameters W.

$$\nabla_W J(W) = \begin{pmatrix} \frac{dJ(W)}{dW_1} \\ \vdots \\ \frac{dJ(W)}{dW_l} \end{pmatrix} \qquad 3.23$$

And the parameters will be updated as,

$$W = W + \alpha \nabla_W J(W) \tag{3.24}$$

Where, α is the small learning rate constant and the policy gradient $\nabla_W J(W)$,

Now the problem is to find the derivative of objective function (Eq:3.22 & 3.23) with respect to W, for which the change in policy and the corresponding reward in each episode is required. According to episodic reinforce algorithm [80,81], it is assumed that the $\pi_W(s,a)$ is differentiable when it is non-zero. Based on which, policy gradient with respect to W can be evaluated by parameterized $\pi_W(s,a)$ and its log likelihood $log(\pi_W(s,a))$. For the ease of understanding, let derive it analytically. Consider y as parameterized policy,

$$y = \pi_W(s,a)$$

and its log likelihood as,

$$z = \log \pi_W(s,a)$$

Then the derivative of log likelihood with respect to W is,

$$\frac{dz}{dW} = \frac{dz}{dy} \cdot \frac{dy}{dW}$$

$$= \frac{\log\big(\pi_W(s,a)\big)}{d(\pi_W(s,a))} \cdot \frac{d(\pi_W(s,a))}{dW}$$

$$= \frac{1}{\pi_W(s,a)} . \nabla_W \pi_W(s,a)$$

$$= \nabla_W \log\big(\pi_W(s,a)\big)$$

Using log identity, $\nabla_w \log(x) = \frac{1}{x} \nabla_w x$,

$$\nabla_W \log\big(\pi_W(s,a)\big) = \frac{\nabla_W \pi_W(s,a)}{\pi_W(s,a)} \tag{3.25}$$

Rearranging above equation,

$$\nabla_W \pi_W(s,a) = \pi_W(s,a) \nabla_W \log\big(\pi_W(s,a)\big) \tag{3.26}$$

Hence, at episode τ,the gradient is written as,

$$\nabla_\tau \pi_\tau(s,a;W) = \pi_\tau(s,a;W) \nabla_\tau \log\big(\pi_\tau(s,a;W)\big) \tag{3.27}$$

Using Eq:3.26, the derivative of objective function (Eq:3.23) become,

$$\nabla J_\tau(W) = \nabla_\tau E_{\pi_\tau}[G_\tau; W] = \sum_\tau \nabla_W \pi_\tau(s,a;W) R_\tau$$

$$= E_{\pi_W}\left[\sum_{t=1}^{T} \pi_t(s_t, a_t; W) R_t \ \nabla_W \log\big(\pi_t(s_t, a_t; W)\big)\right]$$

From Eq: 3.22,

$$\nabla J_\tau(W) = E_{\pi_W}\left[\sum_{t=1}^{T} G_t \ \nabla_W \log\big(\pi_t(s_t, a_t; W)\big)\right] \qquad 3.28$$

Wrapping all above discussed concepts together, the policy gradient driven RL algorithm is written as,

---

$initialize\ policy\ parameter\ W\ arbitrarily$
$terminal\ = False$
$For\ each\ episode\ do$
    $initialize\ S\ \{s_1, a_1, r_1, \ldots . s_T, a_T, r_T\} \sim \pi_W$
    $\log\ policy\ gradients, = []$
    $rewards, R = []$
    $for\ each\ step\ in\ episode\ do$
        $Take\ action\ a, observe\ r\ and\ s'\ using\ policy\ \pi_W(s, a)$
        $R[step] \leftarrow r$
        $Measure\ the\ gradient\ of\ \log\big(\pi(s, a; W)\big)$
        $gradients[step] \leftarrow gradient$
        $update\ current\ state\ ; S \leftarrow S'$
    $for\ gradient, r\ in\ gradients, rewards$
        $Measure\ future\ rewards, G$
        $Update\ parameters\ W \leftarrow W + \ \alpha\ .G.gradient$
$until\ terminal$

The terminal of the algorithm should be chosen based on the task to be performed. For example, in case of cart pole-v0 balancing problem [77], the terminal state is the sum of rewards in episode R >=195 for 100 consecutive episodes.

3.4 SORN for complex classic control tasks

3.4.1 Cartpole pole balancing: Problem statement and the environment

A pole is attached to a cart with an un actuated joint called pivot point (violet square in Figure 3. 10). The center of gravity of the pole is above the pivot point. The agent can control the pole by applying a force +1(Move Left: Green arrow) or -1(Move right: Red arrow) and the task for the agent is to keep the pole upright and prevent it from falling. For each step the pole is up right, the agent gets +1 reward. If the pole deviate 15° from upright position, or the cart moved more than 2.4 units from the center of the environment, then the episode end. To simplify, the agent has to move right if the pole sliding on right side and left if the pole slides towards left to maximize rewards per episode.

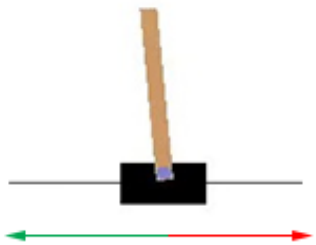


Figure 3. 10: Cartpole Balance problem: Cart and the Pole with Pivot point in blue color

3.4.2 Training Data and Experiment Pipeline

The input data for SORN is four state variables; Cart position [-4.8,4.8], Cart velocity [-infinity, infinity], Pole angle [-24°,24°] and Pole angular velocity [-infinity, infinity].

There are several versions of SORN used for the task and they are classified based on the following aspects:

i) Number of neurons in the reservoir
ii) Number of input units in the reservoir
iii) The input under which networks are simulated
iv) Length of plasticity phase during training
v) Number of initial connections (incoming and outgoing) per neuron

In the task, SORN receive its inputs from the cartpole environment. Therefore, the actual inputs for the agent driven by RL is the reservoir states of SORN. Hence, SORN should be simulated and optimized in such a way that it can encode the dynamics of the environment into its reservoir states. The target for the RL agent is to map these states to corresponding actions (+1, -1) by optimizing the trainable parameters (weights) between SORN and output layer through policy gradient algorithm (*Figure 3. 11*).

To get the relationship (tradeoff) between problem complexity and number of neurons to be used in the reservoir, SORN[23] & [32] with reduced (50) units stabilized by plasticity are also considered. The parameters of the reservoir which are fixed in all the models shown in table 3.1 are ($T_E^{max} = 1.0, T_I^{max} = 0.5$) and the parameters of policy gradient driven reinforcement learning are set to $\gamma = 0.99$ and $\alpha = 0.0001$.

To begin with training, each network must undergo plasticity phase for the length mentioned in the table above. Also note that, during plasticity phase, the linear layer is also optimized subsequently by RL algorithm. But, in previous works [23,32,34], the linear layer was frozen during plasticity letting SORN to learn the spatio temporal patterns in inputs.

In case of cart pole balancing problem, the action performed by the agent at previous time influences its next input (Observation ←→ Action). That is, the sequence of inputs (dynamics of the environment) is dependent on its output (actions). Hence, to preserve the sequence of observations, it is not optimal to freeze the output layer during plasticity phase, otherwise the temporal information in environment state will be lost. However, the randomness is injected at the

beginning of each episode, when the environment reset to random initial state, irrespective of last action performed by the agent at previous episode. Therefore, SORN also has to learn to adapt such alternations in the structure of input.

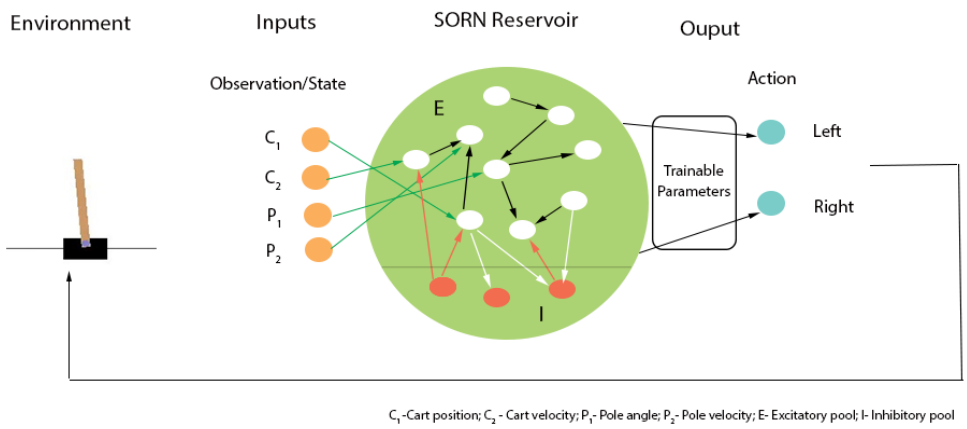


Figure 3. 11: SORN with RL agent training pipeline.

Network type	SORN model	Reservoir size	Input during simulation (Input x strength)	Length of plasticity phase, K (during training)	λ^W	Number of input units
A	SORN [23]	200	*R(10)*x10^4	0	10	4
B	SORN [23]	50	*R(8)*x10^4	2x10^4	30	8
Γ	SORN [23]	50	*R(4)*x10^4	2x10^4	20	4
Δ	SORN [32]	200	*G(10)*	2x10^4	10	4
E	SORN [32]	200	*R(10)* x10^4	1x10^5	20	8
Z	SORN [32]	50	*IS*	1x10^5	30	8
H	SORN [23]	50	*G(8)*	4x10^4	20	8

Table 3.1: The variations in SORN network and training setup: *R(n)* denotes random *n* number of input neurons triggered by the external stimuli. *G(n)* represents input of length *n* from gaussian distribution ($\aleph(0,1)$. *IS* mean that the network is at 'Initialized State' and is not optimized by plasticity under any input.

3.4.3 Results and analysis

The cartpole problem is considered as solved, if the agent (SORN+ RL algorithm) successfully score more than 195 rewards for 100 consecutive episodes [78]. The training length for the agent is fixed at 2x10^5 timesteps during which the SORN undergo plasticity for first K time steps (Table 3.1) driven by the observations. Meanwhile, the RL agent learns to map the highly dynamic network states into actions with an eager to achieve maximum reward at each episode. From the results (*Figure 3. 12*), the agent has achieved the maximum reward at several episodes but failed to produce it consistently in following episodes. However, inferring the achieved results shows the significance of each factors in Table 3.1 towards the adaptability and dynamical stability of SORN exposed to spontaneous sequence of inputs.

Network type	λ_C	Rewards Plasticity Phase					Rewards Training Phase				
		μ	$\tilde{\mu}$	σ^2	*CV*	*SE*	μ	$\tilde{\mu}$	σ^2	*CV*	*SE*
A	7370	-	-	-	-	-	26.51	22.0	14.43	0.544	0
B	505	23.23	20.0	12.81	0.55	1	99.59	91.0	64.11	0.643	20196
Γ	624	29.84	25.0	17.63	0.59	1	72.72	47.0	56.84	0.781	3960
Δ	7278	28.16	23.0	16.62	0.58	0	89.76	89.0	44.92	0.500	1794
E	9656	39.78	24.0	20.18	0.67	0	32.531	27.0	20.79	0.639	4
Z	386	25.84	22.0	14.78	0.57	1	159.40	169.0	44.68	0.280	35644
H	570	26.62	22.0	16.20	0.60	1	137.78	140.0	50.80	0.368	30467

Table 3.2: Performance of SORN models during plasticity phase and training phase: μ – Mean; $\tilde{\mu}$ – Median; σ^2- Standard deviation; CV – Coefficient of Variation (σ^2/μ); SE – Number of Successful Episodes

The results (*Figure 3.12*) shows that the networks (agents) are still far from the state of the art. During plasticity phase (*Figure 3. 12A&B*), none of the networks showed any progress in learning. This is because, SORN undergoes radical changes in its structure during which it exhibits complex dynamics in networks states. Such network behavior destroys the relationship between the environmental states and the consequent actions performed by the RL agent at each time steps. However, during training phase, the network models *B, Γ, H & Δ* has produced considerable performance (*Figure 3. 12C*) while *Z* was exceptional with success rate marginally greater than 1/3.

Furthermore, the network has appeared to produce some consistency in performance during the training phase (*Figure 3. 12A*). The box plot between the time interval 150000 -151000 shows that the 75 % of total rewards lies within upper and lower quartile with a middle quartile/median lies at 200 (Maximum achievable score). The network produced success rate slightly greater than 1/2 with μ and σ^2 of the score lies at 180.57 and 30.62 respectively.

Through covariance measures (Table 3.2), it is found that the networks Z&H produced moderate fluctuations in score during training and have achieved the maximum score often when compared with other models. This meant that the networks have learned to respond to the varying environmental observations with unique and predictable state dynamics, that can be mapped into target actions by the RL agent.

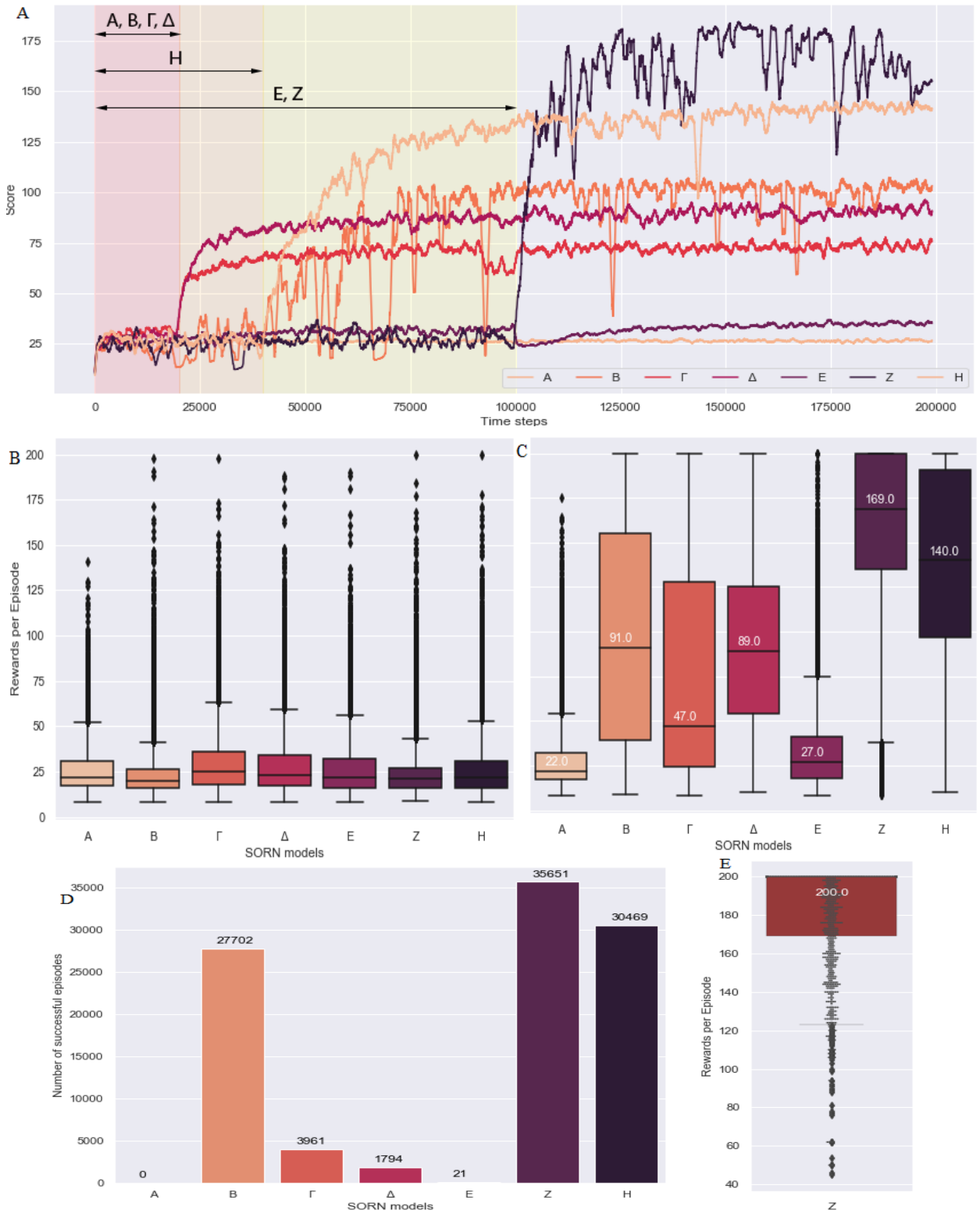


Figure 3. 12:Training results: A) 1000 Average smoothened time series showing the rewards/episode achieved by each model during entire training period. B. Box plot showing the distribution of rewards per episode during plasticity phase. Note that none of the networks achieved maximum score (successful episode) more than once during plasticity phase. C: Distribution of rewards during training phase. D: Count of successful episodes by each network during entire training phase (including plasticity pha se). E: Performance of network Z between 150000- and 151000-time steps.

Linear correlation between the network parameters (*Figure 3. 13*) and their values against the number of successful episodes.

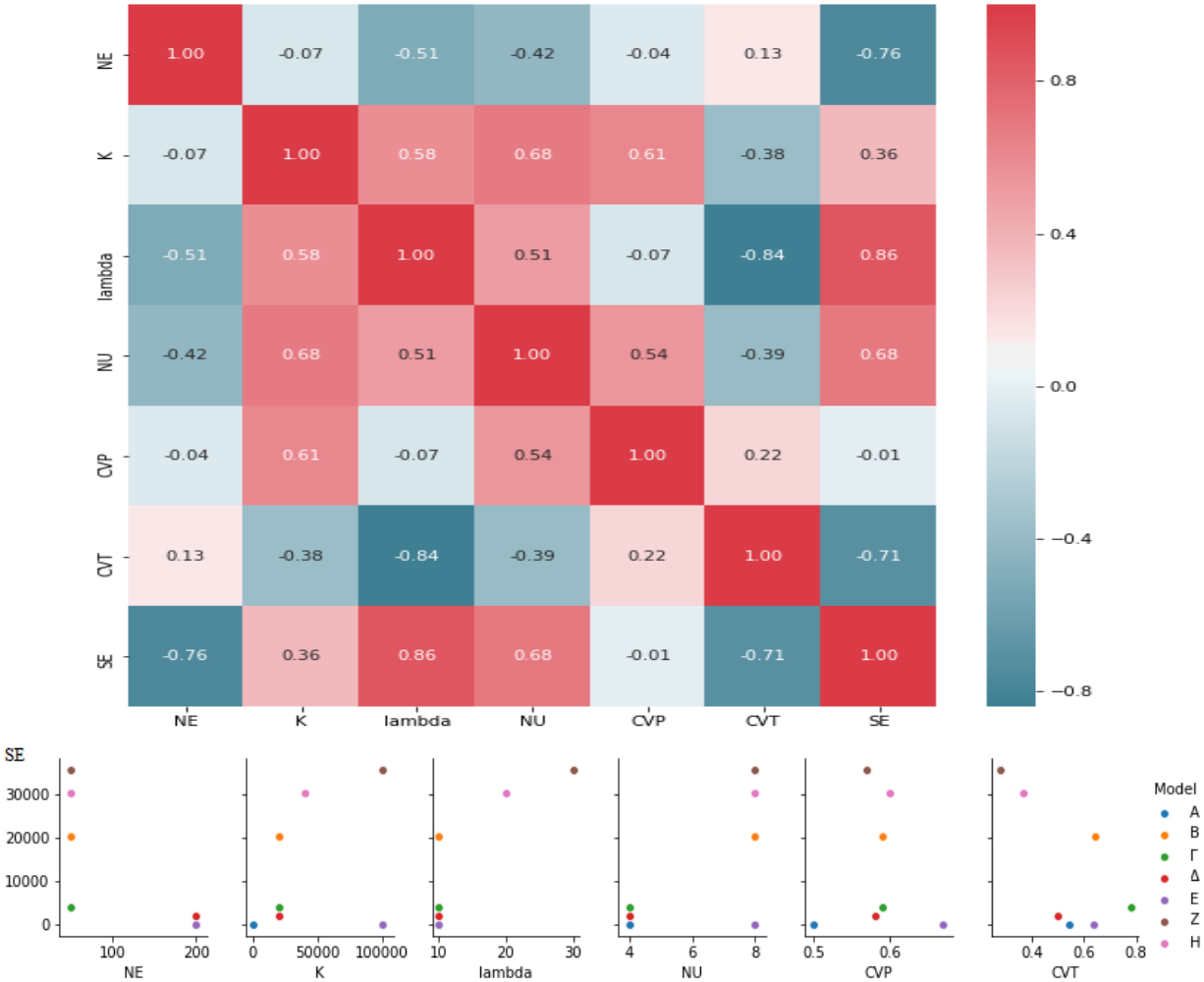


Figure 3. 13: Correlation between parameter variables; Bottom: Parameter variables with respect to number of successful episodes

I suspect that, SORN [32] model *Z* has gained advantages over *H & B* with longer plasticity phase and higher initial connection density. Therefore, the influence of additional plasticity mechanisms and noise over the performance advantages of SORN [32] model against SORN [23] must be investigated. For that, the count of active connections which are optimized by the plasticity mechanisms at the end of plasticity phase during training are considered. From the results (*Figure 3. 14*) of networks with reservoir size 200, *A* and Δ which have similar connection values shows that λ^{w} and λ_{C} have no relationship with network performance, but the plasticity during training phase does. Even though, the network *E* was shaped by the plasticity during training but was not driven by problem-specific input during simulation phase is suspected to be the cause of its failure during training. Similar results are observed from networks with 50 neurons in the reservoir when number of connections considered (*Figure 3. 14*). However, the networks (*Z &H*) which undergone longer plasticity during training and exposed to similar inputs (*H*) or no inputs (*Z*) during simulation phase has outperformed other networks.

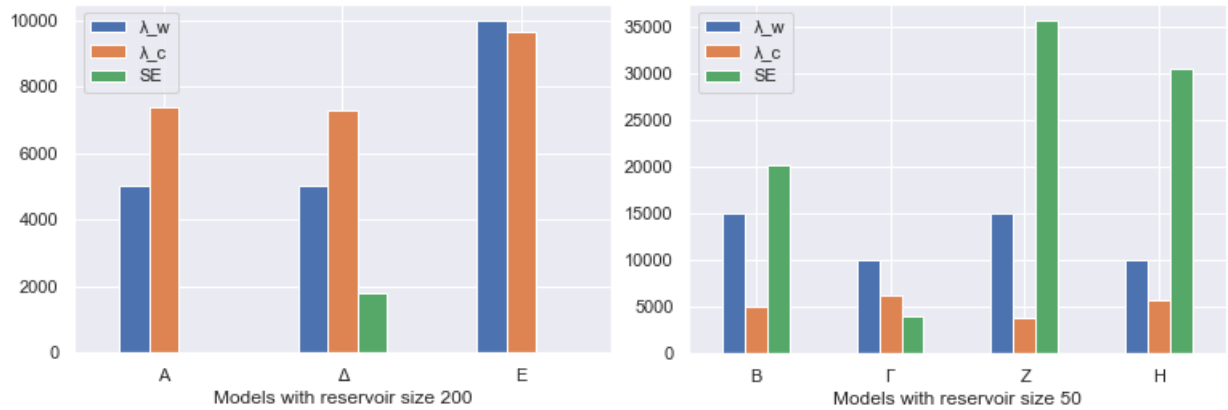


Figure 3. 14: Bar plot showing the relation between number of connections in the reservoir and their overall performance during training; Left: with 200 neurons and Right: With 50 neurons: In figure, number of connections in the reservoir after plasticity phase during simulation λ^w and after plasticity phase during training λ_C. SE represents the number of successful episodes. Note that, for ease of visibility, the λ^w and λ_C are multiplied by a factor 1000 against the scores in y-axis

Overall, the network Z with merely 50 units in the reservoir and 8 input units, simulated by plasticity mechanisms, under small values of noise without random inputs and optimized by plasticity mechanisms during training phase has outperformed the rest of the variations of SORN models and train settings. Therefore, I speculate that, before exposing SORN to training data, it is necessary to simulate them with all five plasticity mechanisms under noise. If the underlying distribution of values in training data is unknown, it is effective to use the network that is simulated input-free.

IV CONCLUSION

a) Research Questions Answered

The goal of this thesis was to reveal the nature of complications in deploying SORN to solve real-time computational problems, to develop methods that mitigate those complications and to prove that this class of brain-inspired networks could achieve competitive performance in sequential decision-making problems. Besides, the properties of this network are also found to be consistent with biological findings. The results show that properly initialized and simulated SORN can be trained by integrating it with variety of ANN optimization algorithms. Furthermore, the training methods developed in this thesis are employed to solve complex sequential decision-making problem. The spiking dynamics of neuronal population with respect to the external stimuli are also investigated and explained.

b) Future Research

Faster Simulation: The computation cost of simulating this network with large number of units (>200) in the reservoir overload CPUs with higher number of floating-point operations. This takes between 3-4 days to simulate this network till stable phase using system with Intel Core i5 processor with 4GB memory. Therefore, there is a need for migrating such operations into GPU, which is cost-effective.

Functional Connectomes: Connectivity structure of clusters of neurons introduced in this thesis opens the possibility to investigate self-organization through graphical models of network-structure-driven activity or activity-driven network structures.

Dimensionality Reduction: One of the reasons for huge fluctuations or inconsistency in the network performance is projecting highly dynamic 50- or 200-dimensional network states into 2-dimensional output space. Hence, it is imperative to perform the principle component analysis on the network states which could decompose it into less components[89] with high variability. This will improve the performance of optimization algorithms while mapping the input with corresponding output. Further, the class of evolutionary algorithms, Neuro-Evolution of Augmented Topologies (NEAT), could also serve this purpose by selectively making connections between neurons with high significance.

Finite vs. Infinite input domain: Certain input parameters in Cartpole environment have continuous and infinite input domain. I speculate that this same network could solve the problems with finite and fully observable input space (discrete state space). For example, Frozen lake path-finding problem.

V BIBLIOGRAPHY:

1. Robert S. Zucker. Neurotransmitter Release and Its Modulation: Reprinted from: L. K. Kaczmarek and I. B. Levitan Neuromodulation; The Biochemical New York: Oxford University Press, Pages 243-263, 1987, https://mcb.berkeley.edu/labs/zucker/PDFs/Zucker_Kaczmarekl.pdf
2. Sharma, N., Classen, J., & Cohen, L. G. (2013). Neural plasticity and its contribution to functional recovery. Handbook of Clinical Neurology, 110, 3–12. https://doi.org/10.1016/B978-0-444-52901-5.00001-0
3. Chang, Y. (2014). Reorganization and plastic changes of the human brain associated with skill learning and expertise. Frontiers in Human Neuroscience, 8. https://doi.org/10.3389/fnhum.2014.00035
4. Blumenfeld-Kutzer, T., Pasternak, O., Dagan, M., & Assaf, Y. (2011). Diffusion MRI of structural brain plasticity induced by a learning and memory task. PLoS ONE, 6(6). https://doi.org/10.1371/journal.pone.0020678
5. Sagi, Y., Tavor, I., Hofstetter, S., Tzur-Moryosef, S., Blumenfeld-Katzir, T., & Assaf, Y. (2012). Learning in the Fast Lane: New Insights into Neuroplasticity. Neuron, 73(6), 1195–1203. https://doi.org/10.1016/j.neuron.2012.01.025
6. Sagi, Y., Tavor, I., Hofstetter, S., Tzur-Moryosef, S., Blumenfeld-Katzir, T., & Assaf, Y. (2012). Learning in the Fast Lane: New Insights into Neuroplasticity. Neuron, 73(6), 1195–1203. https://doi.org/10.1016/j.neuron.2012.01.025
7. Lichtman, J. W., & Colman, H. (2000). Synapse elimination and indelible memory. Neuron. Cell Press. https://doi.org/10.1016/S0896-6273(00)80893-4
8. Hebb, D. O. ; E. A., & Bussey, T. J. (1949). The Organization of Behavior. J. Cogn. Neurosci (Vol. 99, p. 70). John Wiley & Sons 71 Murray. https://doi.org/10.1364/OL.24.000954
9. Malenka, R. C., & Bear, M. F. (2004, September 30). LTP and LTD: An embarrassment of riches. Neuron. https://doi.org/10.1016/j.neuron.2004.09.012
10. Nabavi, S., Fox, R., Proulx, C. D., Lin, J. Y., Tsien, R. Y., & Malinow, R. (2014). Engineering a memory with LTD and LTP. Nature, 511(7509), 348–352. https://doi.org/10.1038/nature13294
11. Hölscher, C. (1999). Synaptic plasticity and learning and memory: LTP and beyond. Journal of Neuroscience Research, 58(1), 62–75. https://doi.org/10.1002/(SICI)1097-4547(19991001)58:1<62::AID-JNR7>3.0.CO;2-G
12. Nicoll, R. A. (2017, January 18). A Brief History of Long-Term Potentiation. Neuron. Cell Press. https://doi.org/10.1016/j.neuron.2016.12.015
13. Abraham, W. C. (2003, April 29). How long will long-term potentiation last? Philosophical Transactions of the Royal Society B: Biological Sciences. Royal Society. https://doi.org/10.1098/rstb.2002.1222
14. Takeuchi, T., Duszkiewicz, A. J., & Morris, R. G. M. (2014, January 5). The synaptic plasticity and memory hypothesis: Encoding, storage and persistence. Philosophical Transactions of the Royal Society B: Biological Sciences. Royal Society. https://doi.org/10.1098/rstb.2013.0288
15. Hao, L., Yang, Z., & Lei, J. (2018). Underlying Mechanisms of Cooperativity, Input Specificity, and Associativity of Long-Term Potentiation Through a Positive Feedback of Local Protein Synthesis. Frontiers in Computational Neuroscience, 12. https://doi.org/10.3389/fncom.2018.00025
16. Purves D, Augustine GJ, Fitzpatrick D, et al., editors. Neuroscience. 2nd edition. Sunderland (MA): Sinauer Associates; 2001. Long-Term Synaptic Potentiation. Available from: https://www.ncbi.nlm.nih.gov/books/NBK10878/
17. McCulloch, W. S., & Pitts, W. (1943). A logical calculus of the ideas immanent in nervous activity. The Bulletin of Mathematical Biophysics, 5(4), 115–133. https://doi.org/10.1007/BF02478259
18. Adrian, E. D. (1914). The all-or-none principle in nerve. Journal of Physiology, 47(6), 460–74. https://doi.org/10.1111/j.1469-185X.1962.tb01046.x

19. Rosenblatt, F. (1958). The perceptron: A probabilistic model for information storage and organization in the brain. Psychological Review, 65(6), 386–408. https://doi.org/10.1037/h0042519
20. Maass, W., & Markram, H. (2004). On the computational power of circuits of spiking neurons. Journal of Computer and System Sciences, 69(4), 593–616. https://doi.org/10.1016/j.jcss.2004.04.001
21. Jaeger, H., & Haas, H. (2004). Harnessing Nonlinearity: Predicting Chaotic Systems and Saving Energy in Wireless Communication. Science, 304(5667), 78–80. https://doi.org/10.1126/science.1091277
22. Le, Q. V., Coates, A., Prochnow, B., & Ng, A. Y. (2011). On Optimization Methods for Deep Learning. Proceedings of The 28th International Conference on Machine Learning (ICML), 265–272. https://doi.org/10.1.1.220.8705
23. Lazar, A. (2009). SORN: a Self-organizing Recurrent Neural Network. Frontiers in Computational Neuroscience, 3. https://doi.org/10.3389/neuro.10.023.2009
24. Abbott, L. F., & Nelson, S. B. (2000). Synaptic plasticity: Taming the beast. Nature Neuroscience, 3(11s), 1178–1183. https://doi.org/10.1038/81453
25. Kessels, H. W., & Malinow, R. (2009, February 12). Synaptic AMPA Receptor Plasticity and Behavior. Neuron. https://doi.org/10.1016/j.neuron.2009.01.015
26. Lüscher, C., & Malenka, R. C. (2012). NMDA receptor-dependent long-term potentiation and long-term depression (LTP/LTD). Cold Spring Harbor Perspectives in Biology, 4(6), 1–15. https://doi.org/10.1101/cshperspect.a005710
27. Gina G. Turrigiano and Sacha B. Nelson. Homeostatic plasticity in the developing nervous system. Nature Reviews, 5:97–107, February 2004.
28. Turrigiano, G. G., & Nelson, S. B. (2000, June 1). Hebb and homeostasis in neuronal plasticity. Current Opinion in Neurobiology. Current Biology Ltd. https://doi.org/10.1016/S0959-4388(00)00091-X4
29. Turrigiano, G. G., & Nelson, S. B. (2004). Homeostatic plasticity in the developing nervous system. Nature Reviews Neuroscience. European Association for Cardio-Thoracic Surgery. https://doi.org/10.1038/nrn1327
30. Turrigiano, G. (2012). Homeostatic synaptic plasticity: Local and global mechanisms for stabilizing neuronal function. Cold Spring Harbor Perspectives in Biology, 4(1). https://doi.org/10.1101/cshperspect.a005736
31. Toyoizumi, T., Kaneko, M., Stryker, M. P., & Miller, K. D. (2014). Modeling the Dynamic Interaction of Hebbian and Homeostatic Plasticity. Neuron, 84(2), 497–510. https://doi.org/10.1016/j.neuron.2014.09.036
32. Zheng, P., Dimitrakakis, C., & Triesch, J. (2013). Network Self-Organization Explains the Statistics and Dynamics of Synaptic Connection Strengths in Cortex. PLoS Computational Biology, 9(1). https://doi.org/10.1371/journal.pcbi.1002848
33. Hartmann, C., Lazar, A., Nessler, B., & Triesch, J. (2015). Where's the Noise? Key Features of Spontaneous Activity and Neural Variability Arise through Learning in a Deterministic Network. PLoS Computational Biology, 11(12). https://doi.org/10.1371/journal.pcbi.1004640
34. Del Papa, B., Priesemann, V., & Triesch, J. (2017). Criticality meets learning: Criticality signatures in a self-organizing recurrent neural network. PLoS ONE, 12(5). https://doi.org/10.1371/journal.pone.0178683
35. Kleberg, F. I., Fukai, T., & Gilson, M. (2014). Excitatory and inhibitory STDP jointly tune feedforward neural circuits to selectively propagate correlated spiking activity. Frontiers in Computational Neuroscience, 8. https://doi.org/10.3389/fncom.2014.00053
36. Caroni, P., Donato, F., & Muller, D. (2012, July). Structural plasticity upon learning: Regulation and functions. Nature Reviews Neuroscience. https://doi.org/10.1038/nrn3258
37. Butz, M., Wörgötter, F., & van Ooyen, A. (2009, May). Activity-dependent structural plasticity. Brain Research Reviews. https://doi.org/10.1016/j.brainresrev.2008.12.023

38. Lamprecht, R., & LeDoux, J. (2004). Structural plasticity and memory. Nature Reviews Neuroscience, 5(1), 45–54. https://doi.org/10.1038/nrn1301
39. Geinisman, Y. (2000). Structural synaptic modifications associated with hippocampal LTP and behavioral learning. Cerebral Cortex (New York, N.Y.: 1991), 10(10), 952–62. https://doi.org/10.1093/cercor/10.10.952
40. Aswolinskiy, W., & Pipa, G. (2015). RM-SORN: a reward-modulated self-organizing recurrent neural network. Frontiers in Computational Neuroscience, 9. https://doi.org/10.3389/fncom.2015.00036
41. Markram, H., and Sakmann, B. (1995). Action potentials propagating back into dendrites triggers changes in efficacy of single-axon synapses between layer V pyramidal cells. Soc. Neurosci. Abstr. 21, 2007.
42. Markram, H., Lübke, J., Frotscher, M., & Sakmann, B. (1997). Regulation of synaptic efficacy by coincidence of postsynaptic APs and EPSPs. Science, 275(5297), 213–215. https://doi.org/10.1126/science.275.5297.213
43. Haas, J. S., Nowotny, T., & Abarbanel, H. D. I. (2006). Spike-Timing-Dependent Plasticity of Inhibitory Synapses in the Entorhinal Cortex. Journal of Neurophysiology, 96(6), 3305–3313. https://doi.org/10.1152/jn.00551.2006
44. Caporale, N., & Dan, Y. (2008). Spike Timing–Dependent Plasticity: A Hebbian Learning Rule. Annual Review of Neuroscience, 31(1), 25–46. https://doi.org/10.1146/annurev.neuro.31.060407.125639
45. Barral, J., & D'Reyes, A. (2016). Synaptic scaling rule preserves excitatory-inhibitory balance and salient neuronal network dynamics. Nature Neuroscience, 19(12), 1690–1696. https://doi.org/10.1038/nn.4415
46. Mayford, M., Siegelbaum, S. A., & Kandel, E. R. (2012). Synapses and memory storage. Cold Spring Harbor Perspectives in Biology, 4(6), 1–18. https://doi.org/10.1101/cshperspect.a005751
47. Brem, A. katharine, Ran, K., & Pascual-leone, A. (2013). Learning and memory. Handbook of Clinical Neurology, 116, 693–737. https://doi.org/10.1016/B978-0-444-53497-2.00055-3
48. Yasumatsu N, Matsuzaki M, Miyazaki T, Noguchi J, Kasai H (2008) Principles of long-term dynamics of dendritic spines. The Journal of Neuroscience 28: 13592–13608.
49. Johansen-Berg, H. (2007, February 20). Structural Plasticity: Rewiring the Brain. Current Biology. https://doi.org/10.1016/j.cub.2006.12.022
50. George, J. B., Abraham, G. M., Rashid, Z., Amrutur, B., & Sikdar, S. K. (2018). Random neuronal ensembles can inherently do context dependent coarse conjunctive encoding of input stimulus without any specific training. Scientific Reports, 8(1). https://doi.org/10.1038/s41598-018-19462-3
51. Turrigiano, G. G. (2008, October 31). The Self-Tuning Neuron: Synaptic Scaling of Excitatory Synapses. Cell. https://doi.org/10.1016/j.cell.2008.10.008
52. Ibata, K., Sun, Q., & Turrigiano, G. G. (2008). Rapid Synaptic Scaling Induced by Changes in Postsynaptic Firing. Neuron, 57(6), 819–826. https://doi.org/10.1016/j.neuron.2008.02.031
53. Houweling, A. R., Bazhenov, M., Timofeev, I., Steriade, M., & Sejnowski, T. J. (2005). Homeostatic synaptic plasticity can explain post-traumatic epileptogenesis in chronically isolated neocortex. Cerebral Cortex, 15(6), 834–845. https://doi.org/10.1093/cercor/bhh184
54. Song, S., Sjöström, P. J., Reigl, M., Nelson, S., & Chklovskii, D. B. (2005). Highly nonrandom features of synaptic connectivity in local cortical circuits. In PLoS Biology(Vol. 3, pp. 0507–0519). https://doi.org/10.1371/journal.pbio.0030068
55. Feldmeyer, D., Lübke, J., Silver, R. A., & Sakmann, B. (2002). Synaptic connections between layer 4 spiny neurone-layer 2/3 pyramidal cell pairs in juvenile rat barrel cortex: Physiology and anatomy of interlaminar signalling within a cortical column. Journal of Physiology, 538(3), 803–822. https://doi.org/10.1113/jphysiol.2001.012959

56. 29. Arellano, J. I. (2007). Ultrastructure of dendritic spines: correlation between synaptic and spine morphologies. Frontiers in Neuroscience, 1(1), 131–143. https://doi.org/10.3389/neuro.01.1.1.010.2007
57. Song, S., Sjöström, P. J., Reigl, M., Nelson, S., & Chklovskii, D. B. (2005). Highly nonrandom features of synaptic connectivity in local cortical circuits. In PLoS Biology (Vol. 3, pp. 0507–0519). https://doi.org/10.1371/journal.pbio.0030068
58. 31. Sarid, L., Bruno, R., Sakmann, B., Segev, I., & Feldmeyer, D. (2007). Modeling a layer 4-to-layer 2/3 module of a single column in rat neocortex: Interweaving in vitro and in vivo experimental observations. Proceedings of the National Academy of Sciences, 104(41), 16353–16358. https://doi.org/10.1073/pnas.0707853104
59. 32. Loewenstein, Y., Kuras, A., & Rumpel, S. (2011). Multiplicative Dynamics Underlie the Emergence of the Log-Normal Distribution of Spine Sizes in the Neocortex In Vivo. Journal of Neuroscience, 31(26), 9481–9488. https://doi.org/10.1523/JNEUROSCI.6130-10.2011
60. Latuske, P., Toader, O., & Allen, K. (2015). Interspike Intervals Reveal Functionally Distinct Cell Populations in the Medial Entorhinal Cortex. Journal of Neuroscience, 35(31), 10963–10976. https://doi.org/10.1523/JNEUROSCI.0276-15.2015
61. Reich, D. S., Mechler, F., Purpura, K. P., & Victor, J. D. (2000). Interspike intervals, receptive fields, and information encoding in primary visual cortex. The Journal of Neuroscience : The Official Journal of the Society for Neuroscience, 20(5), 1964–74. https://doi.org/0270-6474/00/201964-11
62. Spanne, A., Geborek, P., Bengtsson, F., & Jarntell, H. (2014). Spike generation estimated from stationary spike trains in a variety of neurons in vivo. Frontiers in Cellular Neuroscience, 8. https://doi.org/10.3389/fncel.2014.00199
63. Stiefel, K. M., Englitz, B., & Sejnowski, T. J. (2013). Origin of intrinsic irregular firing in cortical interneurons. Proceedings of the National Academy of Sciences, 110(19), 7886–7891. https://doi.org/10.1073/pnas.1305219110
64. Dayan P & Abbott LF. Theoretical Neuroscience: Computational and Mathematical Modeling of Neural Systems. Cambridge, Massachusetts: The MIT Press; 2001. ISBN 0-262-04199-5
65. Rieke F, Warland D, de Ruyter van Steveninck R, Bialek W. Spikes: Exploring the Neural Code. Cambridge, Massachusetts: The MIT Press; 1999. ISBN 0-262-68108-0
66. Eden, U. T., & Kramer, M. A. (2010). Drawing inferences from Fano factor calculations. Journal of Neuroscience Methods, 190(1), 149–152. https://doi.org/10.1016/j.jneumeth.2010.04.012
67. Song, S., Sjöström, P. J., Reigl, M., Nelson, S., & Chklovskii, D. B. (2005). Highly nonrandom features of synaptic connectivity in local cortical circuits. In PLoS Biology(Vol. 3, pp. 0507–0519). https://doi.org/10.1371/journal.pbio.0030068
68. Feldmeyer, D., Lübke, J., Silver, R. A., & Sakmann, B. (2002). Synaptic connections between layer 4 spiny neurone-layer 2/3 pyramidal cell pairs in juvenile rat barrel cortex: Physiology and anatomy of interlaminar signalling within a cortical column. Journal of Physiology, 538(3), 803–822. https://doi.org/10.1113/jphysiol.2001.012959
69. 29. Arellano, J. I. (2007). Ultrastructure of dendritic spines: correlation between synaptic and spine morphologies. Frontiers in Neuroscience, 1(1), 131–143. https://doi.org/10.3389/neuro.01.1.1.010.2007
70. Song, S., Sjöström, P. J., Reigl, M., Nelson, S., & Chklovskii, D. B. (2005). Highly nonrandom features of synaptic connectivity in local cortical circuits. In PLoS Biology (Vol. 3, pp. 0507–0519). https://doi.org/10.1371/journal.pbio.0030068
71. 31. Sarid, L., Bruno, R., Sakmann, B., Segev, I., & Feldmeyer, D. (2007). Modeling a layer 4-to-layer 2/3 module of a single column in rat neocortex: Interweaving in vitro and in vivo experimental observations. Proceedings of the National Academy of Sciences, 104(41), 16353–16358. https://doi.org/10.1073/pnas.0707853104

72. 32. Loewenstein, Y., Kuras, A., & Rumpel, S. (2011). Multiplicative Dynamics Underlie the Emergence of the Log-Normal Distribution of Spine Sizes in the Neocortex In Vivo. Journal of Neuroscience, 31(26), 9481–9488. https://doi.org/10.1523/JNEUROSCI.6130-10.2011
73. French, A. S., & Holden, A. V. (1971). Alias-free sampling of neuronal spike trains. Kybernetik, 8(5), 165–171. https://doi.org/10.1007/BF00291117
74. Maass, W. (2015). To Spike or Not to Spike: That Is the Question. Proceedings of the IEEE, 103(12), 2219–2224. https://doi.org/10.1109/JPROC.2015.2496679
75. Fano, U. (1947). Ionization yield of radiations. II. the fluctuations of the number of ions. Physical Review, 72(1), 26–29. https://doi.org/10.1103/PhysRev.72.26
76. Heeger, D. (2000). Poisson Model of Spike Generation. Handout, University of Sanford, 5, 1–13. https://doi.org/10.1.1.37.6580
77. Kaelbling, L. P., Littman, M. L., & Moore, A. W. (1996). Reinforcement learning: A survey. Journal of Artificial Intelligence Research, 4, 237–285. https://doi.org/10.1613/jair.301
78. Ng, A. (2012). notes 12 : Reinforcement Learning and Control. Machine Learning, 1–15. https://doi.org/10.1007/978-1-4613-1683-1
79. Hochreiter, S., Younger, A. S., & Conwell, P. R. (2001). Learning to learn using gradient descent. In Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics) (Vol. 2130, pp. 87–94). Springer Verlag. https://doi.org/10.1007/3-540-44668-0_13
80. Williams, R. J. (1988). Toward a theory of reinforcement-learning connectionist systems. Technical Report NU-CCS-88-3, Northeastern University, College of Computer Science.
81. Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. Machine Learning 8:229-256.
82. Barto, A. G., Sutton, R. S. and Anderson, C. (1983), 'Neuron-like adaptive elements that can solve difficult learning control problems', IEEE Transactions on Systems, 5 Man, and Cybernetics 13, 834–846. http://www.cs.ualberta.ca/~sutton/papers/barto-sutton-anderson-83.pdf.gz
83. Bertschinger, N., & Natschläger, T. (2004). Real-time computation at the edge of chaos in recurrent neural networks. Neural Computation, 16(7), 1413–1436. https://doi.org/10.1162/089976604323057443
84. Boedecker, J., Obst, O., Lizier, J. T., Mayer, N. M., & Asada, M. (2012). Information processing in echo state networks at the edge of chaos. Theory in Biosciences, 131(3), 205–213. https://doi.org/10.1007/s12064-011-0146-8
85. Lee, D., Port, N. L., Kruse, W., & Georgopoulos, A. P. (1998). Variability and correlated noise in the discharge of neurons in motor and parietal areas of the primate cortex. The Journal of Neuroscience, 18(3), 1161–70. https://doi.org/citeulike-article-id:90465
86. Smith, M. A., & Kohn, A. (2008). Spatial and Temporal Scales of Neuronal Correlation in Primary Visual Cortex. Journal of Neuroscience, 28(48), 12591–12603. https://doi.org/10.1523/JNEUROSCI.2929-08.2008
87. Cohen, M. R., & Newsome, W. T. (2008). Context-Dependent Changes in Functional Circuitry in Visual Area MT. Neuron, 60(1), 162–173. https://doi.org/10.1016/j.neuron.2008.08.007
88. Maimon, G., & Assad, J. A. (2009). Beyond Poisson: Increased Spike-Time Regularity across Primate Parietal Cortex. Neuron, 62(3), 426–440. https://doi.org/10.1016/j.neuron.2009.03.021
89. Kobak, D., Brendel, W., Constantinidis, C., Feierstein, C. E., Kepecs, A., Mainen, Z. F., … Machens, C. K. (2016). Demixed principal component analysis of neural population data. ELife, 5(April 2016). https://doi.org/10.7554/eLife.10989

VI SUPPORTING INFORMATION

S1: SEQUENCE PREDICTION TASKS: ACTUAL PERFORMANCE RESULTS

Figure S1.1: SORN [23] simulated with random gaussian inputs ($\aleph(0,1)$ for $1x10^6$ time steps;

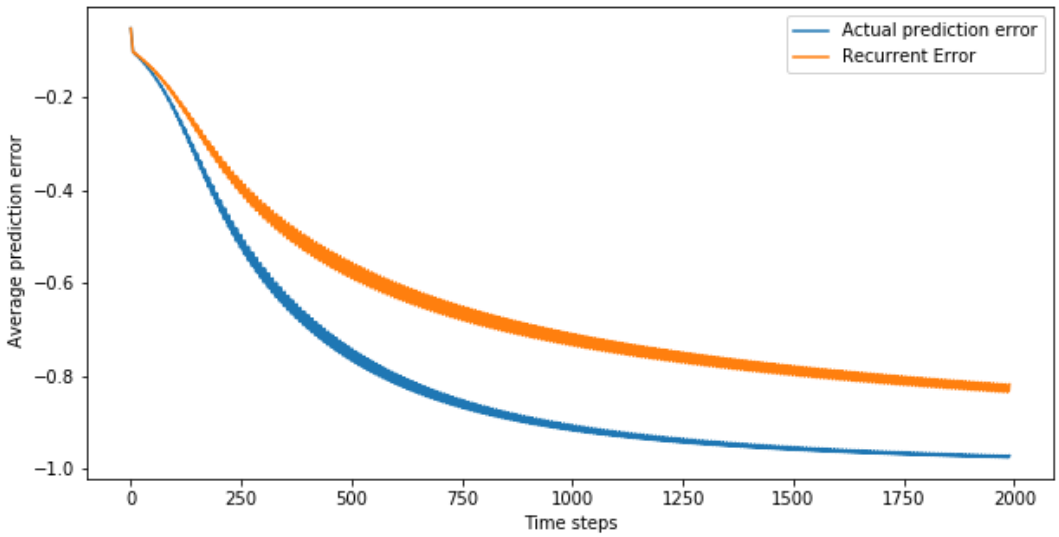


Figure S1.2: SORN [23] simulated with random strong inputs for $1x10^6$ time steps;

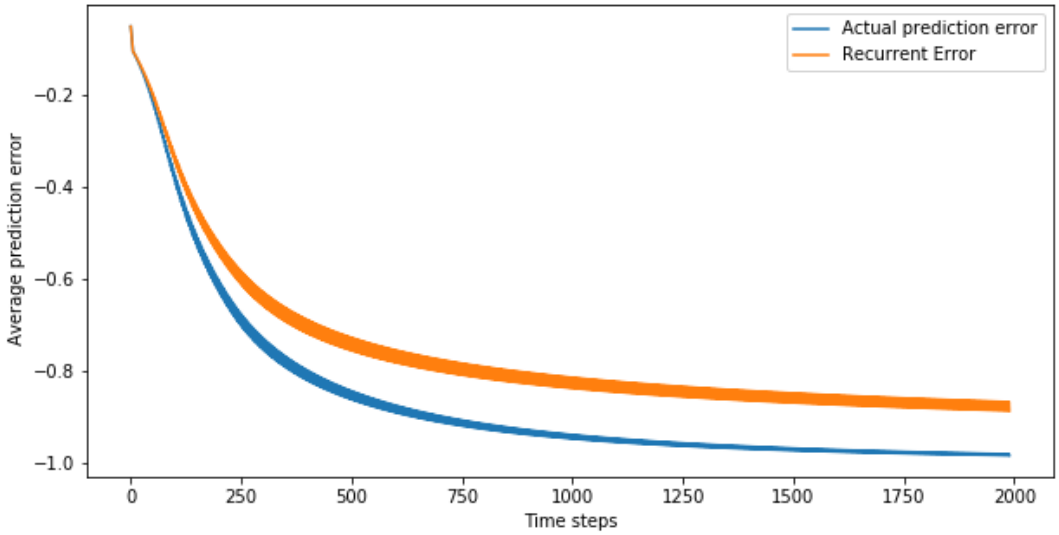


Figure S1.3: SORN [32] simulated with random gaussian inputs ($\aleph(0,1)$ and noise $\xi_E(0,0.04)$ for 3x10^6 time steps;

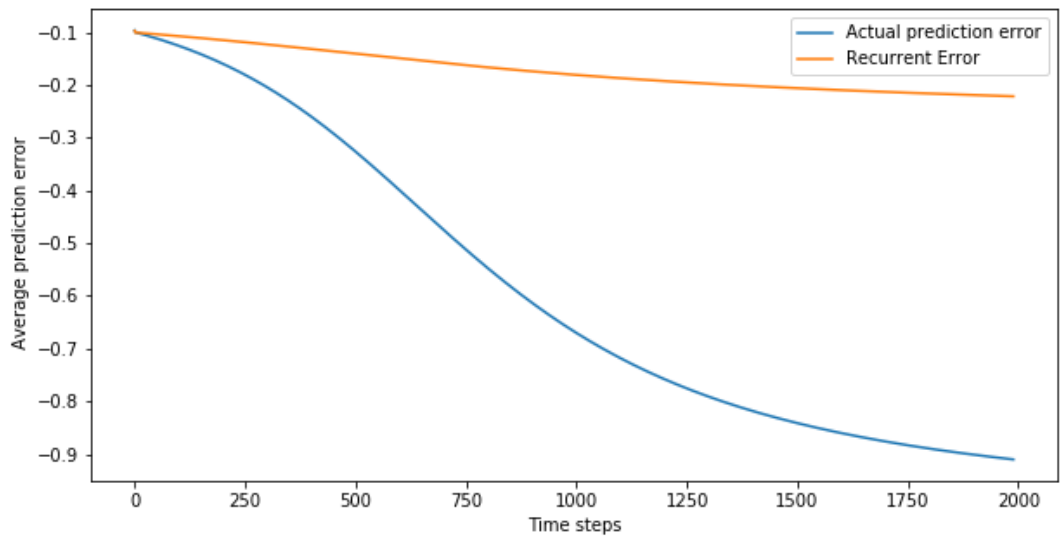
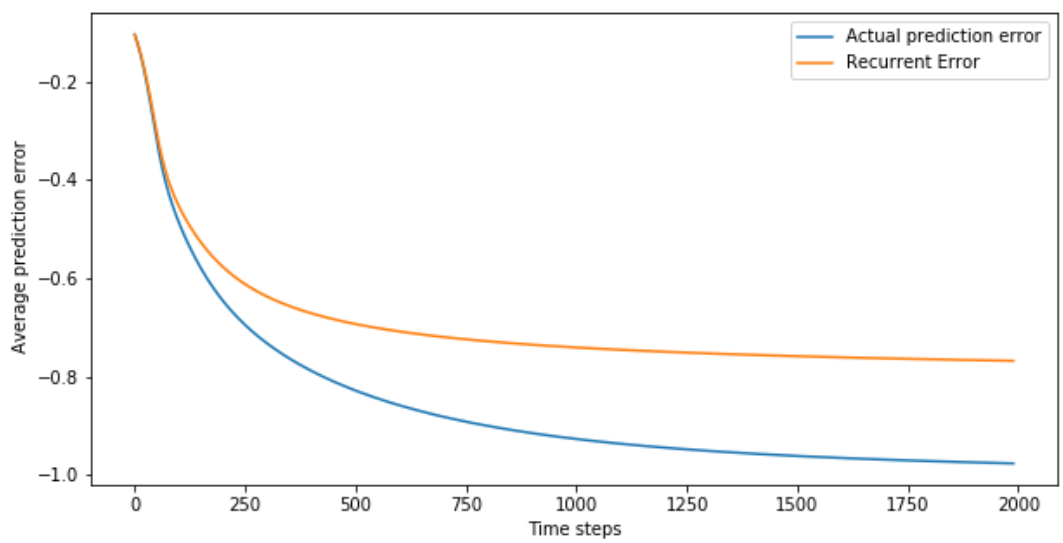


Figure S1.4: SORN [32] simulated without inputs but under noise $\xi_E(0,0.04)$ for 2x10^6 time steps



S2: CARTPOLE BALANCING PROBLEM: ACTUAL PERFORMANCE RESULTS

Figure S2.1: SORN [23] strong inputs 200 units in the reservoir: 4 input units in the reservoir without plasticity phase during training

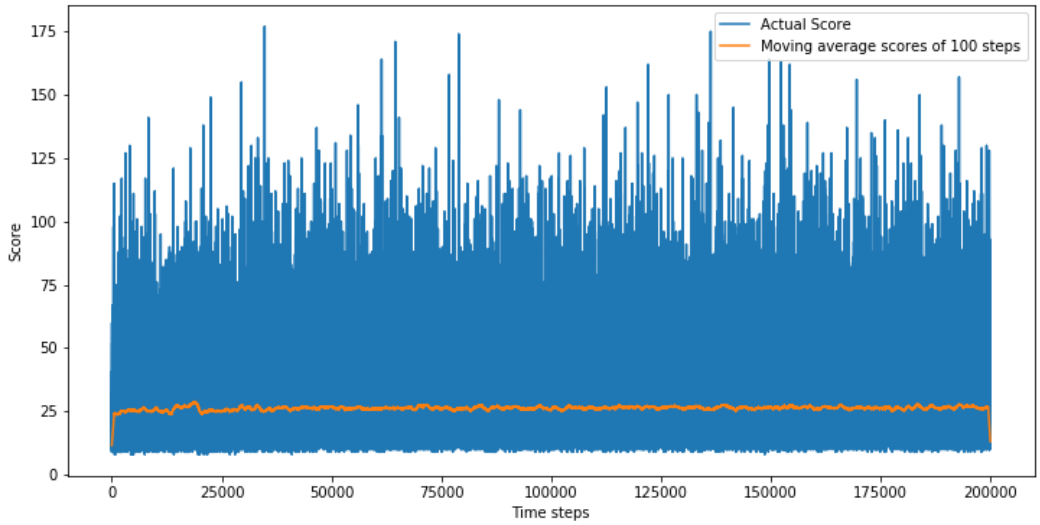


Figure S2.2: SORN [23] strong inputs 50 units in the reservoir: 4 input units with $2x10^5$ time steps of plasticity phase during training

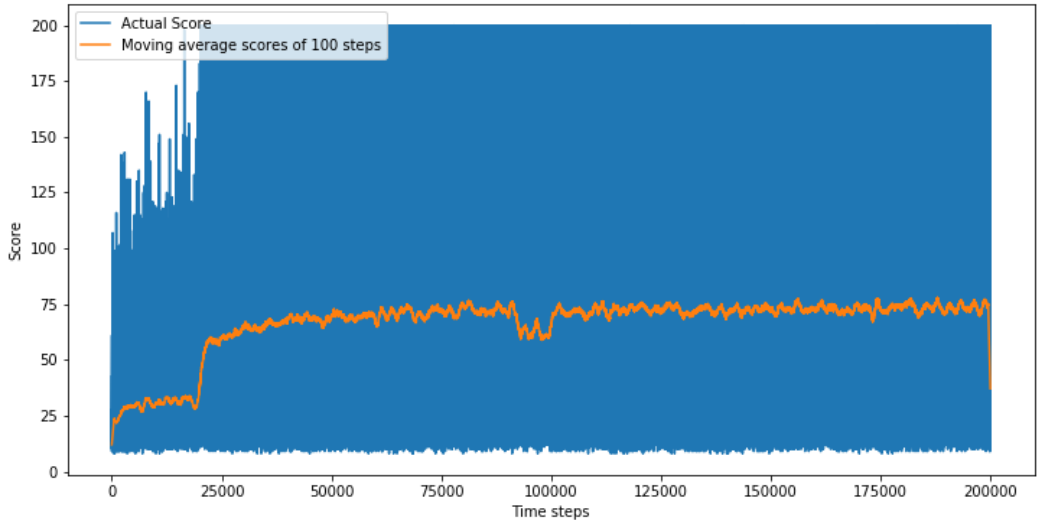


Figure S2.3: SORN [23] strong inputs 50 units in the reservoir: 8 input units with $2x10^5$ time steps of plasticity phase during training.

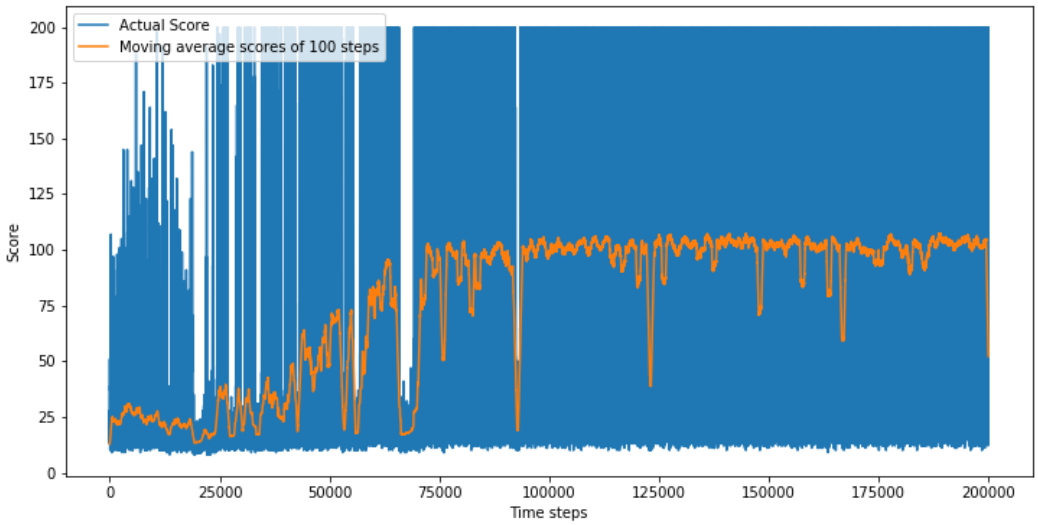


Figure S2.4: SORN [32] gaussian inputs 50 units in the reservoir: 8 input units with $1x10^6$ time steps of plasticity phase during training

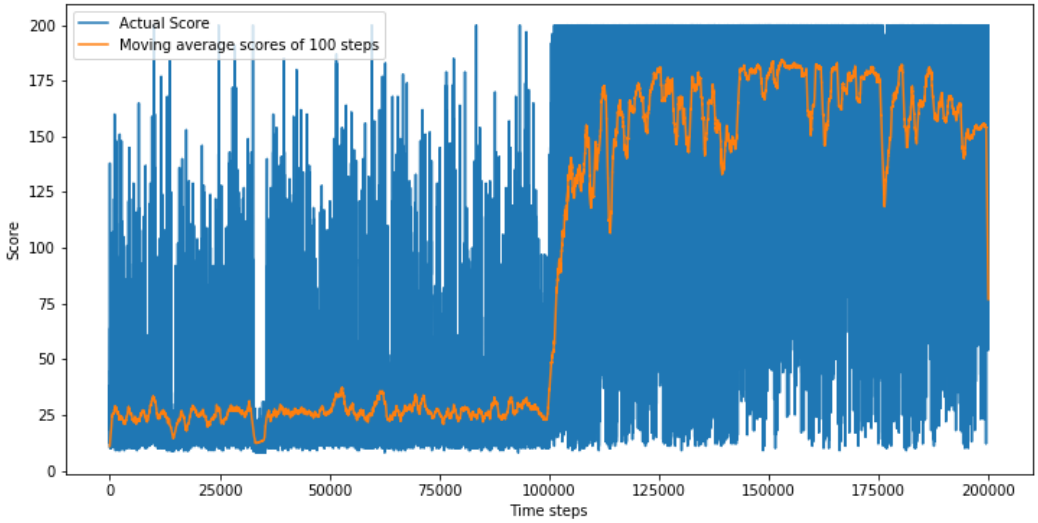


Figure S2.5: SORN [23] gaussian inputs 200 units in the reservoir: 4 input units with $2x10^5$ time steps of plasticity phase during training

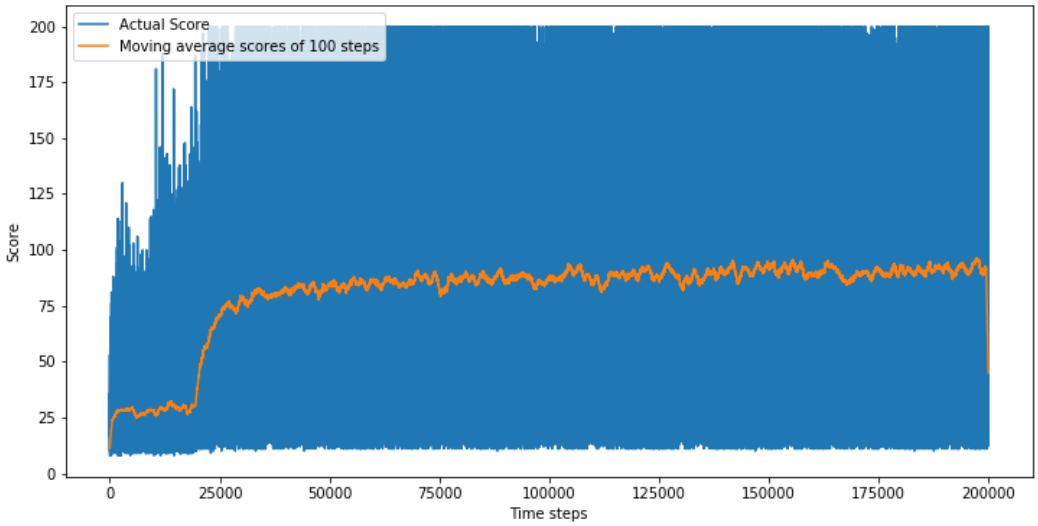


Figure S2.6: SORN [32] gaussian inputs 200 units in the reservoir: 8 input units with $1x10^6$ time steps of plasticity phase during training

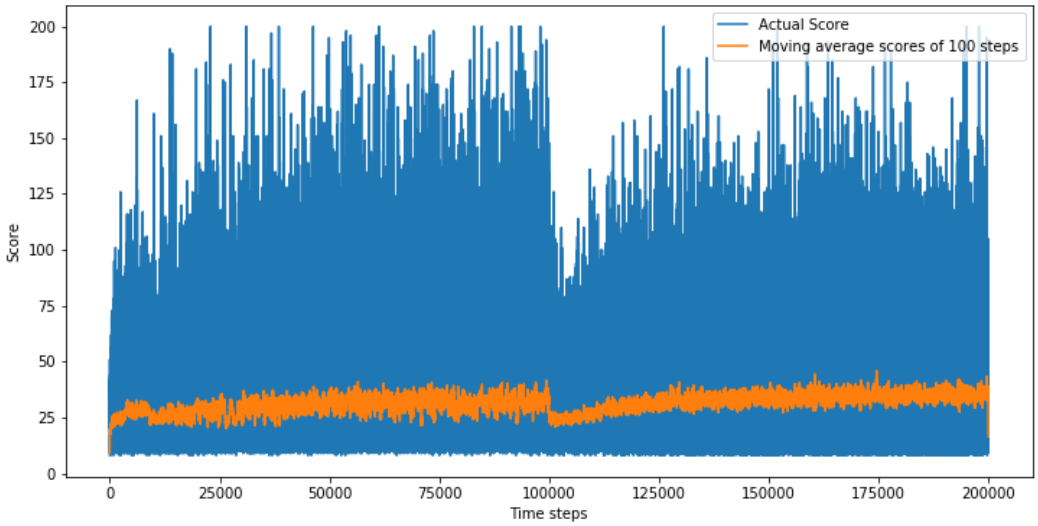
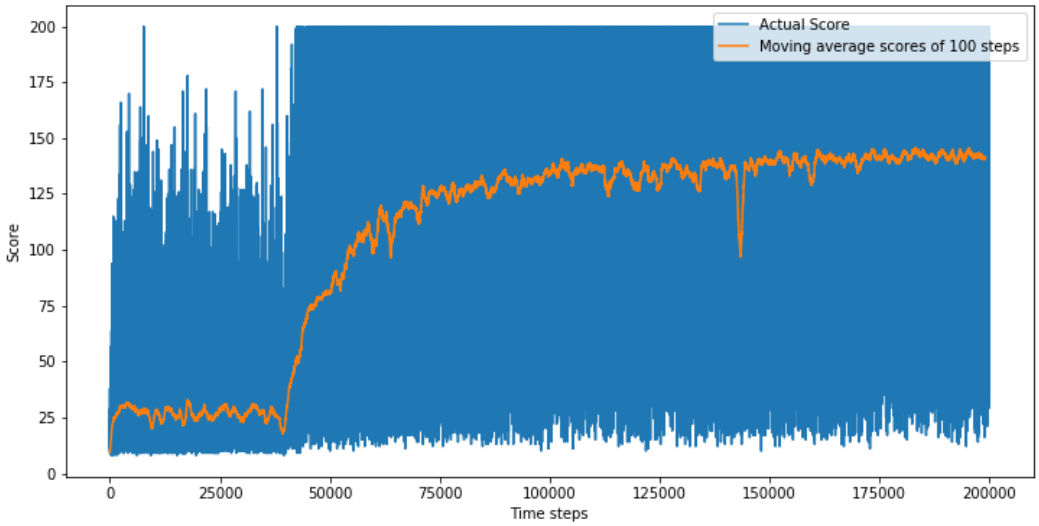



Figure S2.7: SORN [32] gaussian inputs 50 units in the reservoir: 8 input units with $4x10^5$ time steps of plasticity phase during training